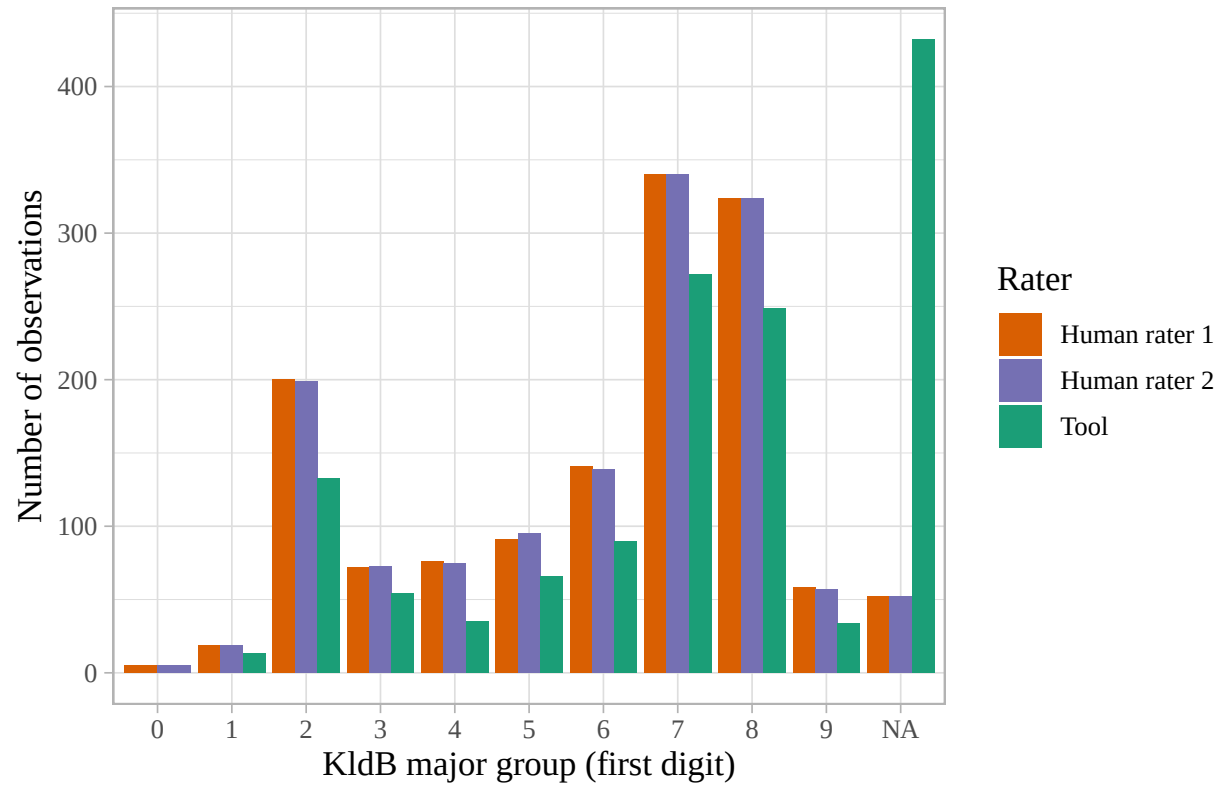
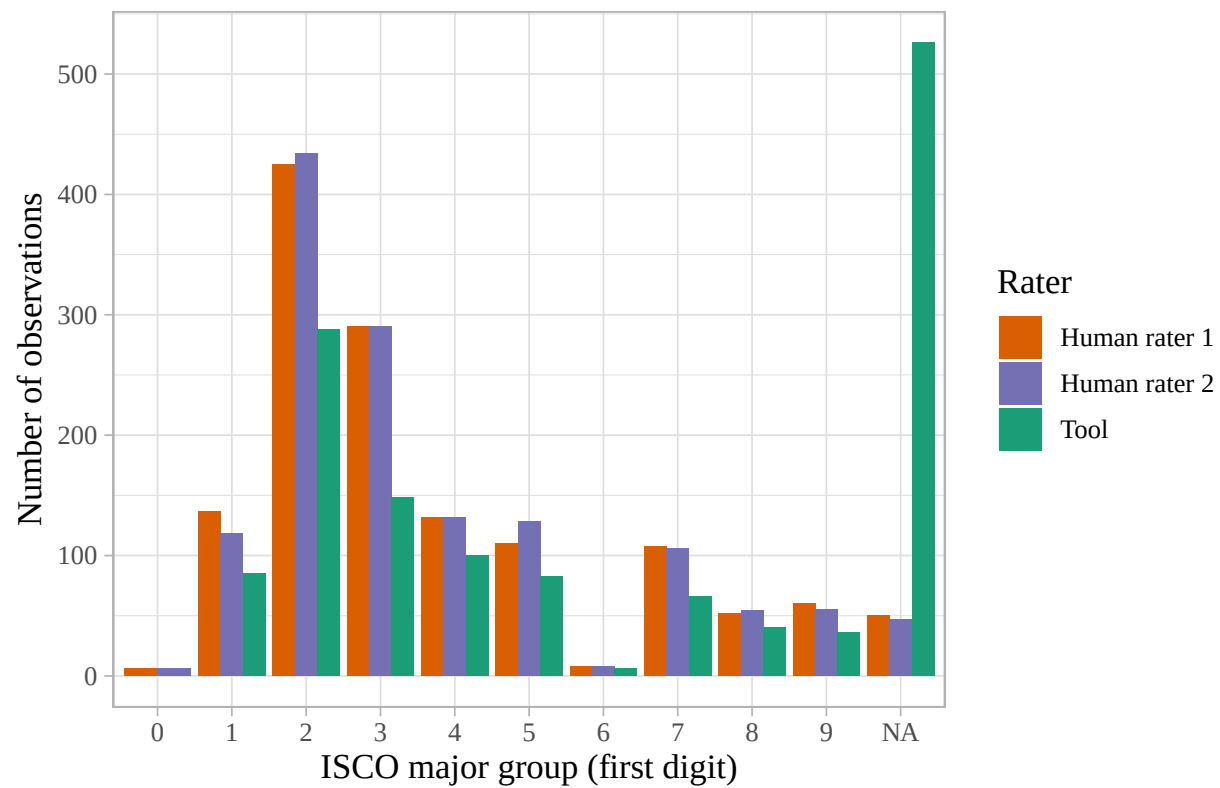


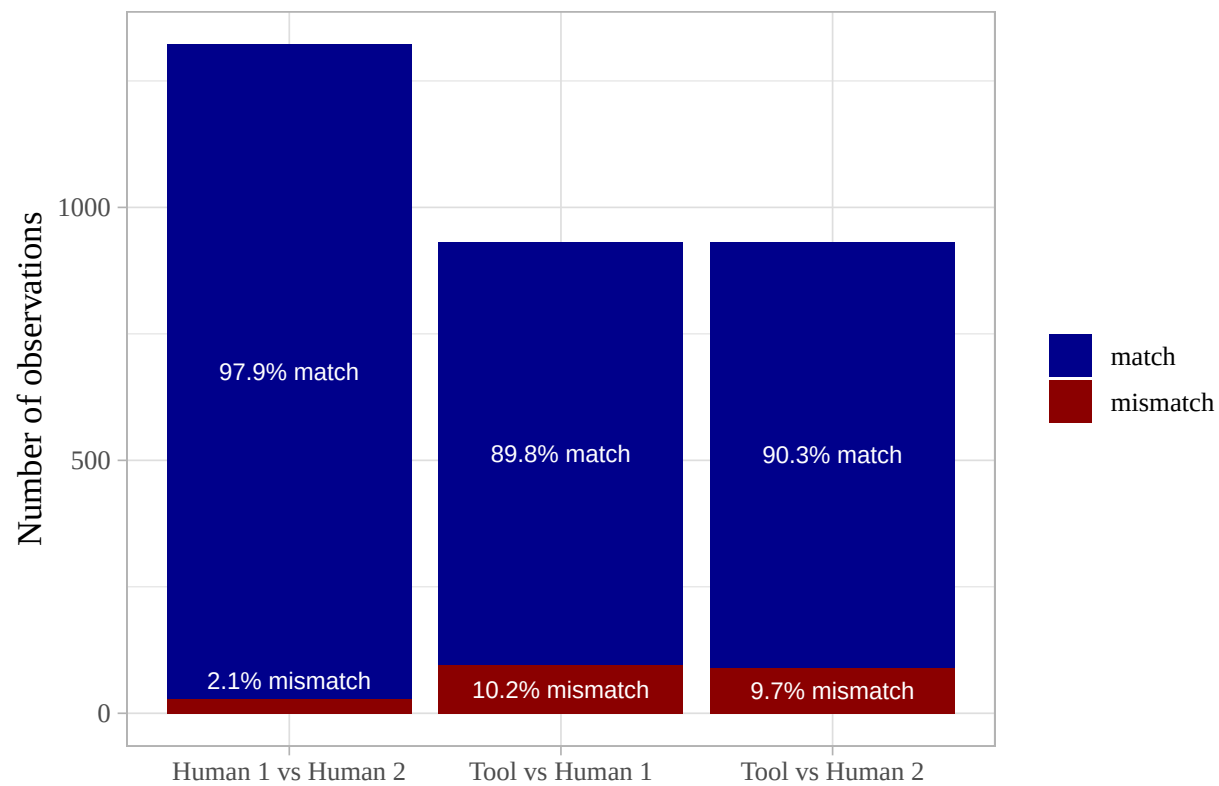
model_basis_kldb_all

```
## [1] "C:/Users/jeroe/Uni_C/noisy_measurement/src/data_preparation.R"
```

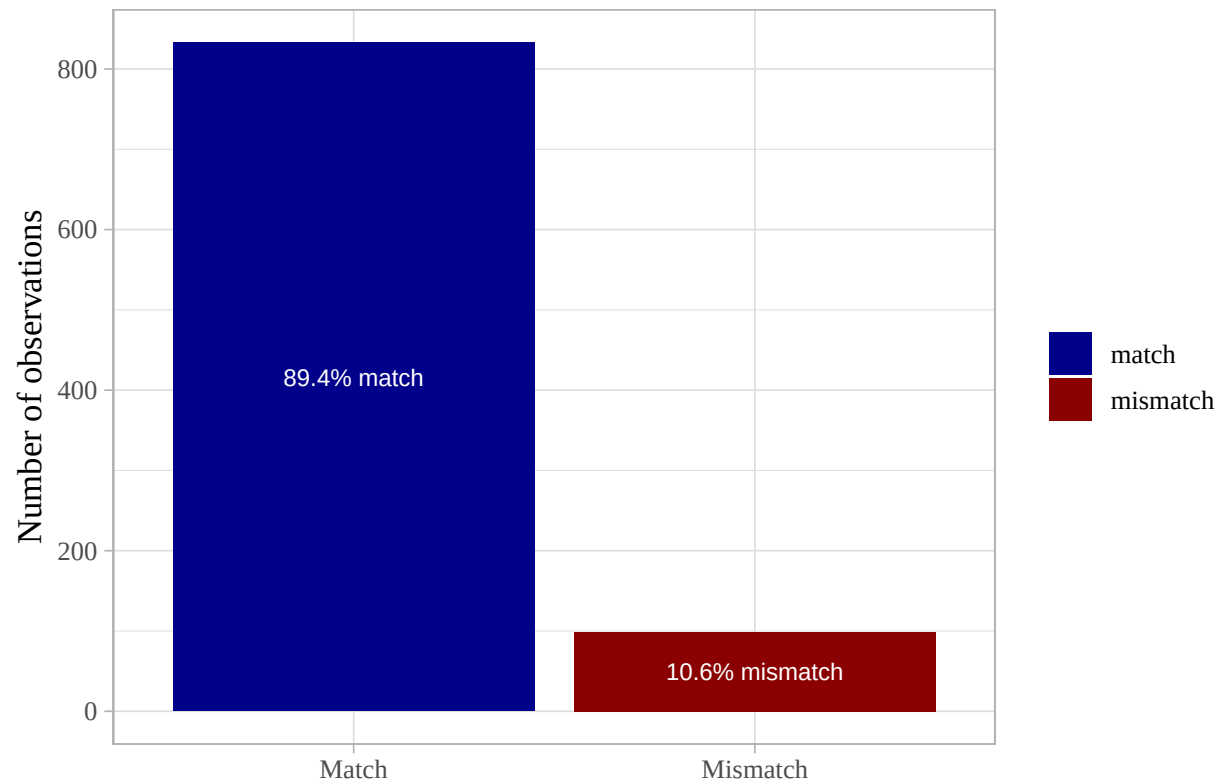




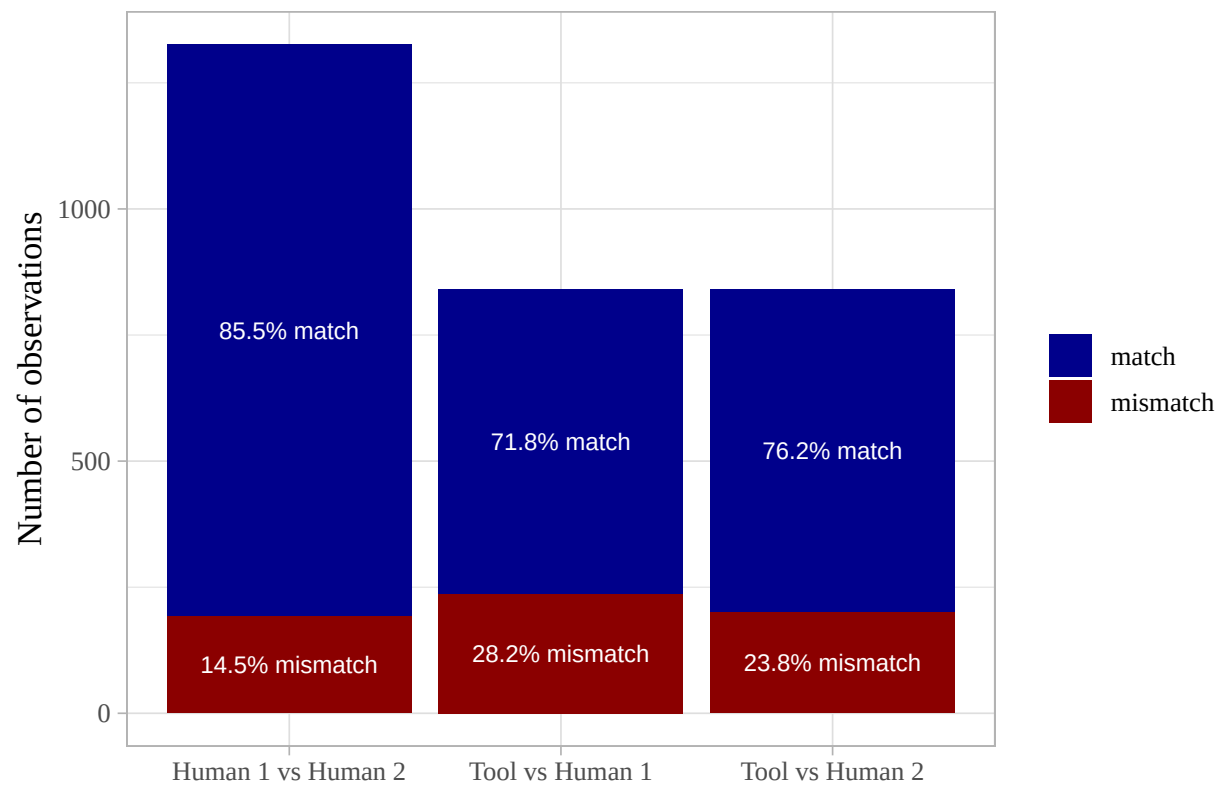
KldB: Pairwise agreement on the first digit



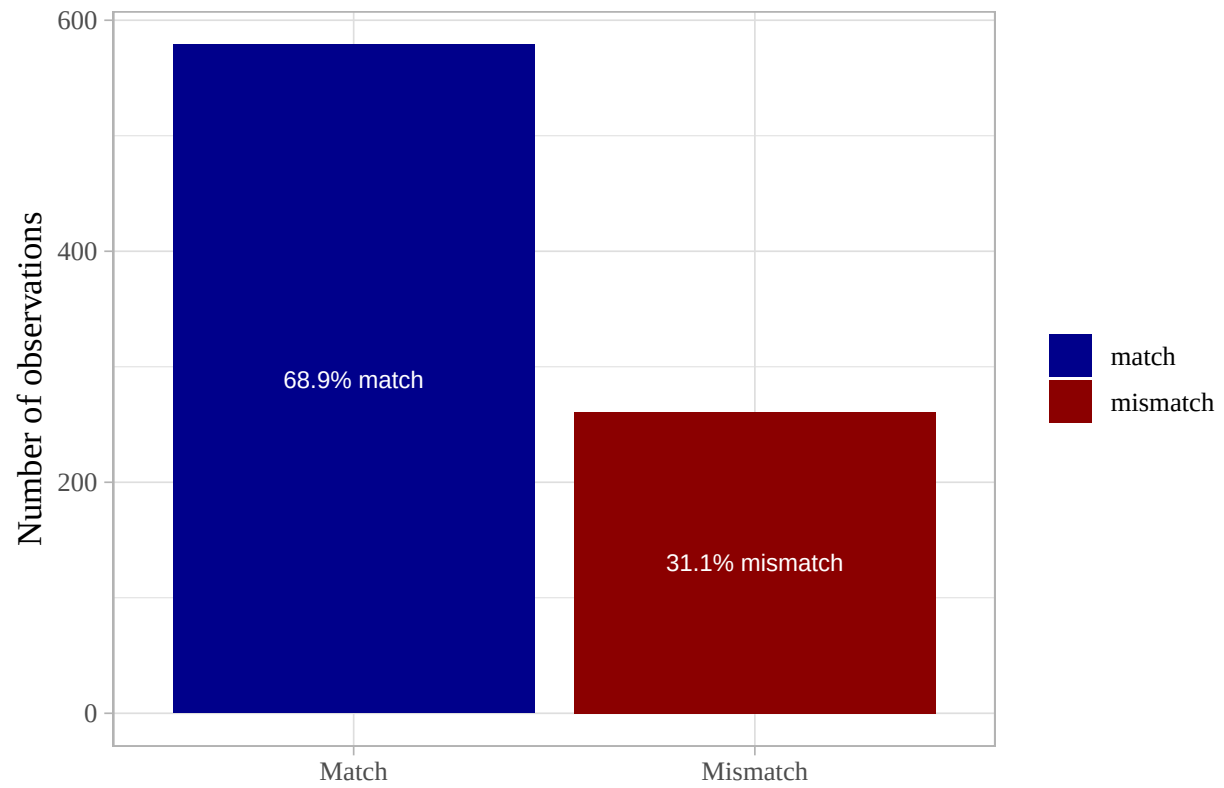
KldB: Agreement across all three sources on the first digit

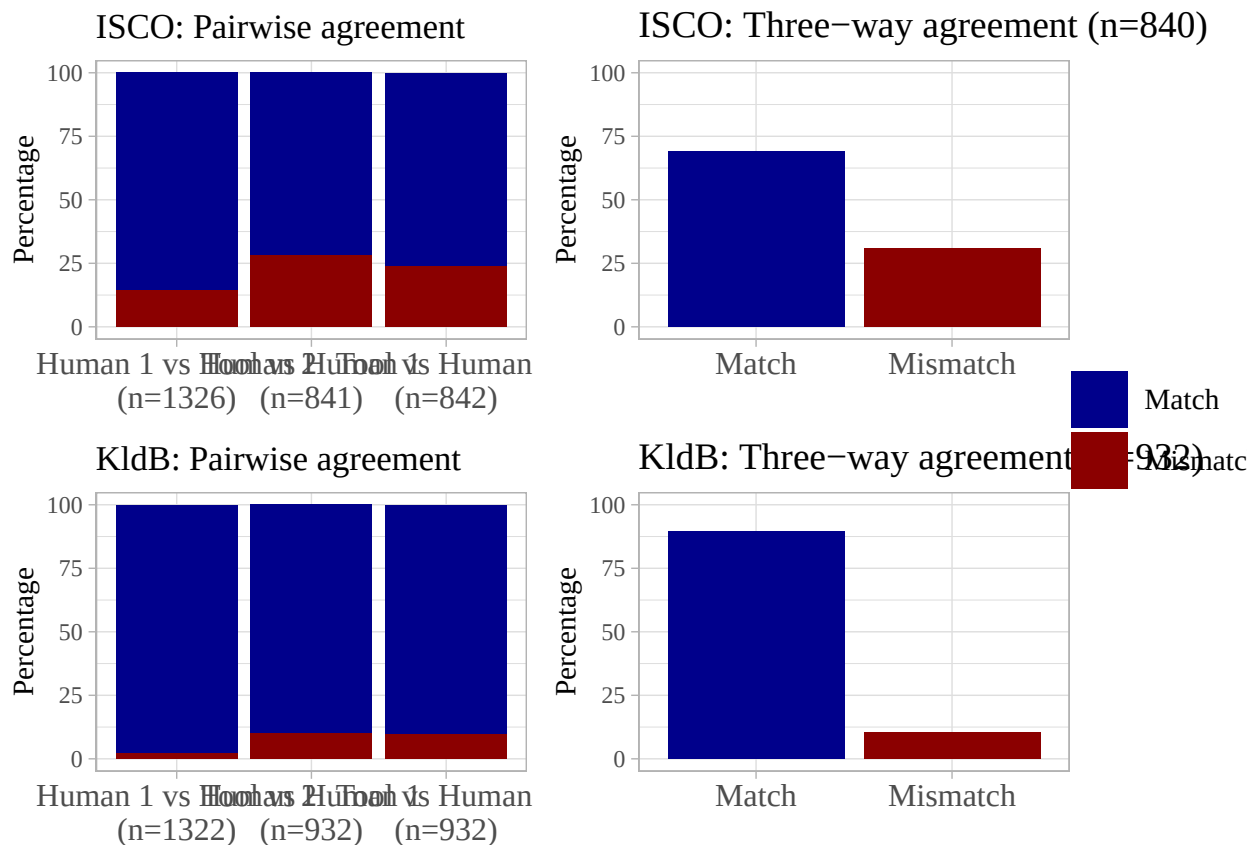


ISCO: Pairwise agreement on the first digit



ISCO: Agreement across all three sources on the first digit





```
# Packages
library(data.table)
library(dplyr)
library(tidyr)
library(cmdstanr)

# keep only relevant columns
kldb_data <- data_basis %>%
  select(item_id, kldb_tool_1st, res1_kldb_1st, res2_kldb_1st)

#Rename columns
setnames(
  data_basis,
  old = c("kldb_tool_1st", "res1_kldb_1st", "res2_kldb_1st"),
  new = c("kldb_tool", "kldb_human1", "kldb_human2")
)

head(data_basis)
```

```
##      kldb_tool_vorlaufig kldb_tool_mit_anfniveau res1_kldb res2_kldb isco_tool_vorlaufig isco_tool_mit_anfniveau
##      <char>           <char>           <char>   <char>           <char>           <char>
## 1:      81883         81883         81883     81883           2433
## 2:      52112         52112         52112     52112           8322
```

```

## 3:      <NA>      <NA>      83131      83131      <NA>
## 4:      23411     23411     23411     23411     9329
## 5:      72134     72134     72132     72132     2412
## 6:      25104     25104     27224     27224     2144
##      res1_isco res2_isco kldb_infas_num      kldb_infas_str isco_infas_nu
##      <char>      <char>      <char>      <char>      <cha
## 1:      2433      2433      81883      [81883] Pharmazie (s.s.T.) - Spezialist      24
## 2:      8322      8322      52112      [52112] Berufskraftfahrer(Pers./PKW.)-Fachkraft      83
## 3:      3412      3412      83131      [83131] Heilerziehungspflege, Sonderpaed. -Helfer      34
## 4:      7322      7322      23412      [23412] Drucktechnik - Fachkraft      73
## 5:      4312      4312      72132      [72132] Versicherungskaufleute - Fachkraft      43
## 6:      2149      2144      27224      [27224] Konstruktion und Geraetebau - Experte      21
##
##
## 1: [2433] Akademische und vergleichbare Fachkräfte im Bereich Vertrieb (Technik und Medizin, ohne In
## 2:      [8322] Personenkraftwagen-, T
## 3:      [3412] Nicht
## 4:
## 5:      [4312] Bürokräfte in der St
## 6:
##      kldb_tool kldb_human1 kldb_human2 isco_tool_1st res1_isco_1st res2_isco_1st item_id
##      <char>      <char>      <char>      <char>      <char>      <char>      <int>
## 1:      8      8      8      2      2      2      1
## 2:      5      5      5      8      8      8      2
## 3:      <NA>      8      8      <NA>      3      3      3
## 4:      2      2      2      9      7      7      4
## 5:      7      7      7      2      4      4      5
## 6:      2      2      2      2      2      2      6

```

```
#### 1) Select only Kldb, as character
```

```

kldb_data <- data_basis %>%
  dplyr::select(item_id, kldb_human1, kldb_human2, kldb_tool) %>%
  mutate(across(c(kldb_human1, kldb_human2, kldb_tool), as.character))

```

```
### Remove all lines with NAs
```

```

kldb_data <- kldb_data %>%
  filter(!is.na(kldb_human1) & !is.na(kldb_human2) & !is.na(kldb_tool))

```

```
# First:
```

```
### Keep only two most common classes: "7" and "8"
```

```

kldb_data <- kldb_data %>%
  filter(kldb_human1 == "7" | kldb_human1 == "8") %>%
  filter(kldb_human2 == "7" | kldb_human2 == "8") %>%
  filter(kldb_tool == "7" | kldb_tool == "8")

```

```
# 1) Number of rows where all three classify as "7"
```

```

count_all_7 <- kldb_data %>%
  dplyr::filter(kldb_human1 == "7", kldb_human2 == "7", kldb_tool == "7") %>%
  nrow()

```

```
# 2) Number of rows where all three classify as "8"
```

```

count_all_8 <- kldb_data %>%
  dplyr::filter(kldb_human1 == "8", kldb_human2 == "8", kldb_tool == "8") %>%

```



```

nrow()

# 3) Number of rows where at least one value is "7" and at least one value is "8"
count_mixed_7_and_8 <- kldb_data %>%
  dplyr::rowwise() %>%
  dplyr::filter(
    any(c(kldb_human1, kldb_human2, kldb_tool) == "7") &
    any(c(kldb_human1, kldb_human2, kldb_tool) == "8")
  ) %>%
  nrow()

# 4) Number of rows where minimum one element is neither "7" nor "8" - should be 0
count_other <- kldb_data %>%
  dplyr::rowwise() %>%
  dplyr::filter(
    any(!c(kldb_human1, kldb_human2, kldb_tool) %in% c("7", "8"))
  ) %>%
  nrow()

# Show results
count_all_7          # 225

## [1] 225
count_all_8          # 242

## [1] 242
count_mixed_7_and_8 # 4

## [1] 4
count_other          # 0

## [1] 0

# Reindex item_id (1, 2, 3,...)
kldb_data <- kldb_data %>%
  dplyr::mutate(item_id = dplyr::row_number())

# Collect all observed classes
kldb_levels <- kldb_data %>%
  dplyr::select(kldb_human1, kldb_human2, kldb_tool) %>%
  unlist(use.names = FALSE) %>%
  unique() %>%
  sort() %>%
  as.character() %>%
  as.vector()

#head(kldb_data)

# K, number of distinct classes
K <- length(kldb_levels)
stopifnot(K >= 2)

# Helper: map original labels to 1..K using kldb_levels for stan compatibility
to_index <- function(v) as.integer(factor(v, levels = kldb_levels))

```

```

# 1) Map original labels to 1..K
kldb_data_idx <- kldb_data %>%
  dplyr::mutate(
    kldb_human1 = to_index(kldb_human1),
    kldb_human2 = to_index(kldb_human2),
    kldb_tool   = to_index(kldb_tool)
  )

# 2) Long format (now without NAs)
long_data <- kldb_data_idx %>%
  dplyr::transmute(item_id, r1 = kldb_human1, r2 = kldb_human2, r3 = kldb_tool) %>%
  tidyr::pivot_longer(cols = starts_with("r"), names_to = "rater", values_to = "y") %>%
  dplyr::mutate(rater_id = as.integer(factor(rater, levels = c("r1", "r2", "r3")))) %>%
  dplyr::arrange(item_id, rater_id)

# Robust 1..I indexing
item_ids <- sort(unique(long_data$item_id))
long_data$item_id_idx <- as.integer(factor(long_data$item_id, levels = item_ids))

I <- length(item_ids) # number of items (people interviewed)
J <- 3L # number of raters
N <- nrow(long_data) # number of observations (I * J if no missing)

cat("Items after filtering:", I, "\n") #354

## Items after filtering: 471
cat("Observations:", N, "\n") #3*354=1062

## Observations: 1413
cat("Classes:", K, "\n") #2

## Classes: 2

# Packages
library(data.table)
library(dplyr)
library(tidyr)
library(cmdstanr)

# Fully neutral Stan data
stan_data <- list(
  I = I,
  J = J,
  K = K,
  N = N,
  ii = as.integer(long_data$item_id_idx),
  jj = as.integer(long_data$rater_id),
  y = as.integer(long_data$y),

  # No data-driven information
  alpha = rep(1.0, K),

```

```

    beta = matrix(1.0, nrow = K, ncol = K)
)

# 3) Write Stan model to file
stan_file <- "dawid_skene_kldb.stan"
writeLines(con = stan_file, text = '
    data {
        int<lower=2> K;                // number of categories
        int<lower=1> I;                // items
        int<lower=1> J;                // raters
        int<lower=1> N;                // number of observed ratings
        array[N] int<lower=1, upper=I> ii; // item index per observation
        array[N] int<lower=1, upper=J> jj; // rater index per observation
        array[N] int<lower=1, upper=K> y;  // observed category per observation

        vector<lower=0>[K] alpha;        // prior for pi
        array[K] vector<lower=0>[K] beta; // Dirichlet priors for theta rows, vary by true class
    }

    parameters {
        simplex[K] pi;                  // prevalence over true classes
        array[J, K] simplex[K] theta;   // confusion rows: for rater j and true class k
    }

    transformed parameters {
        // log_q_z[i, k] = log p(z_i = k | pi, theta) up to a constant
        array[I] vector[K] log_q_z;

        // Initialize with log pi
        for (i in 1:I) {
            log_q_z[i] = log(pi);
        }

        // Accumulate per observation
        for (n in 1:N) {
            int i = ii[n];
            int j = jj[n];
            int y_n = y[n];
            for (k in 1:K) {
                // Add log categorical pmf of y_n under theta[j, k, :]
                log_q_z[i, k] += categorical_lpmf(y_n | theta[j, k]);
            }
        }
    }

    model {
        // Priors
        pi ~ dirichlet(alpha);
        for (j in 1:J)
            for (k in 1:K)
                theta[j, k] ~ dirichlet(beta[k]);

        // Marginalized likelihood

```

```

    for (i in 1:I) {
      target += log_sum_exp(log_q_z[i]);
    }
  }

  generated quantities {
    array[I] simplex[K] q_z;
    for (i in 1:I) {
      vector[K] lq = log_q_z[i];
      q_z[i] = softmax(lq);
    }
  }
}')

# 4) Compile and sample with cmdstanr
mod <- cmdstan_model(stan_file, cpp_options = list(stan_threads = TRUE))

fit_basis <- mod$sample(
  data = stan_data,
  seed = 1234,
  chains = 4,
  parallel_chains = 4,
  threads_per_chain = 1,
  iter_warmup = 1000,
  iter_sampling = 4000,
  refresh = 200,
  adapt_delta = 0.99
)

```

```

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 1 Iteration:   200 / 5000 [ 4%] (Warmup)
## Chain 4 Iteration:   200 / 5000 [ 4%] (Warmup)
## Chain 2 Iteration:   200 / 5000 [ 4%] (Warmup)
## Chain 3 Iteration:   200 / 5000 [ 4%] (Warmup)
## Chain 1 Iteration:   400 / 5000 [ 8%] (Warmup)
## Chain 4 Iteration:   400 / 5000 [ 8%] (Warmup)
## Chain 3 Iteration:   400 / 5000 [ 8%] (Warmup)
## Chain 2 Iteration:   400 / 5000 [ 8%] (Warmup)
## Chain 4 Iteration:   600 / 5000 [12%] (Warmup)
## Chain 1 Iteration:   600 / 5000 [12%] (Warmup)
## Chain 4 Iteration:   800 / 5000 [16%] (Warmup)
## Chain 3 Iteration:   600 / 5000 [12%] (Warmup)
## Chain 2 Iteration:   600 / 5000 [12%] (Warmup)
## Chain 1 Iteration:   800 / 5000 [16%] (Warmup)
## Chain 3 Iteration:   800 / 5000 [16%] (Warmup)
## Chain 2 Iteration:   800 / 5000 [16%] (Warmup)
## Chain 1 Iteration:  1000 / 5000 [20%] (Warmup)
## Chain 1 Iteration:  1001 / 5000 [20%] (Sampling)
## Chain 4 Iteration:  1000 / 5000 [20%] (Warmup)

```

```

## Chain 4 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 2 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 2 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 3 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 3 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 1 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 2 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 4 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 3 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 1 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 2 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 3 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 4 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 2 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 1 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 3 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 2 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 1 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 4 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 2 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 3 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 1 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 2 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 3 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 4 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 2 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 1 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 3 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 2 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 4 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 1 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 3 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 2 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 1 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 4 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 2 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 3 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 1 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 2 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 3 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 4 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 2 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 1 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 3 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 2 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 4 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 1 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 3 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 2 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 1 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 3 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 2 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 4 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 2 Iteration: 4200 / 5000 [ 84%] (Sampling)

```

```

## Chain 3 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 1 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 2 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 4 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 3 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 1 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 2 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 3 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 4 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 1 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 2 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 3 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 2 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 2 finished in 55.4 seconds.
## Chain 1 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 4 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 3 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 1 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 4 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 3 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 1 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 3 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 4 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 1 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 3 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 3 finished in 65.0 seconds.
## Chain 1 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 1 finished in 66.2 seconds.
## Chain 4 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 4 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 4 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 4 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 4 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 4 finished in 83.7 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 67.6 seconds.
## Total execution time: 83.9 seconds.

# print number of rhats above 1.1
s <- fit_basis$summary()
s$rhat <- as.numeric(s$rhat)

subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))[, c("variable", "rhat", "ess_bulk", "ess_tail")]

## # A tibble: 0 x 4
## # i 4 variables: variable <chr>, rhat <dbl>, ess_bulk <dbl>, ess_tail <dbl>

# print string if no rhats above 1.1
if (nrow(subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))) == 0) {
  cat("No R-hat values above 1.1\n")
}

## No R-hat values above 1.1

```

```

fit_chains_lower <- mod$sample(
  data = stan_data,
  seed = 1234,
  chains = 2,
  parallel_chains = 4,
  threads_per_chain = 1,
  iter_warmup = 1000,
  iter_sampling = 4000,
  refresh = 200,
  adapt_delta = 0.99
)

```

Running MCMC with 2 chains, at most 4 in parallel, with 1 thread(s) per chain...

```

##
## Chain 1 Iteration:    1 / 5000 [  0%] (Warmup)
## Chain 2 Iteration:    1 / 5000 [  0%] (Warmup)
## Chain 1 Iteration:   200 / 5000 [  4%] (Warmup)
## Chain 2 Iteration:   200 / 5000 [  4%] (Warmup)
## Chain 1 Iteration:   400 / 5000 [  8%] (Warmup)
## Chain 2 Iteration:   400 / 5000 [  8%] (Warmup)
## Chain 1 Iteration:   600 / 5000 [ 12%] (Warmup)
## Chain 2 Iteration:   600 / 5000 [ 12%] (Warmup)
## Chain 1 Iteration:   800 / 5000 [ 16%] (Warmup)
## Chain 2 Iteration:   800 / 5000 [ 16%] (Warmup)
## Chain 1 Iteration:  1000 / 5000 [ 20%] (Warmup)
## Chain 1 Iteration:  1001 / 5000 [ 20%] (Sampling)
## Chain 2 Iteration:  1000 / 5000 [ 20%] (Warmup)
## Chain 2 Iteration:  1001 / 5000 [ 20%] (Sampling)
## Chain 1 Iteration:  1200 / 5000 [ 24%] (Sampling)
## Chain 2 Iteration:  1200 / 5000 [ 24%] (Sampling)
## Chain 1 Iteration:  1400 / 5000 [ 28%] (Sampling)
## Chain 2 Iteration:  1400 / 5000 [ 28%] (Sampling)
## Chain 2 Iteration:  1600 / 5000 [ 32%] (Sampling)
## Chain 1 Iteration:  1600 / 5000 [ 32%] (Sampling)
## Chain 2 Iteration:  1800 / 5000 [ 36%] (Sampling)
## Chain 1 Iteration:  1800 / 5000 [ 36%] (Sampling)
## Chain 2 Iteration:  2000 / 5000 [ 40%] (Sampling)
## Chain 1 Iteration:  2000 / 5000 [ 40%] (Sampling)
## Chain 2 Iteration:  2200 / 5000 [ 44%] (Sampling)
## Chain 2 Iteration:  2400 / 5000 [ 48%] (Sampling)
## Chain 1 Iteration:  2200 / 5000 [ 44%] (Sampling)
## Chain 2 Iteration:  2600 / 5000 [ 52%] (Sampling)
## Chain 1 Iteration:  2400 / 5000 [ 48%] (Sampling)
## Chain 2 Iteration:  2800 / 5000 [ 56%] (Sampling)
## Chain 1 Iteration:  2600 / 5000 [ 52%] (Sampling)
## Chain 2 Iteration:  3000 / 5000 [ 60%] (Sampling)
## Chain 1 Iteration:  2800 / 5000 [ 56%] (Sampling)
## Chain 2 Iteration:  3200 / 5000 [ 64%] (Sampling)
## Chain 2 Iteration:  3400 / 5000 [ 68%] (Sampling)
## Chain 1 Iteration:  3000 / 5000 [ 60%] (Sampling)
## Chain 2 Iteration:  3600 / 5000 [ 72%] (Sampling)
## Chain 1 Iteration:  3200 / 5000 [ 64%] (Sampling)
## Chain 2 Iteration:  3800 / 5000 [ 76%] (Sampling)
## Chain 1 Iteration:  3400 / 5000 [ 68%] (Sampling)

```

```

## Chain 2 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 2 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 1 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 2 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 1 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 2 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 1 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 2 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 2 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 2 finished in 53.5 seconds.
## Chain 1 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 1 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 1 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 1 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 1 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 1 finished in 63.7 seconds.
##
## Both chains finished successfully.
## Mean chain execution time: 58.6 seconds.
## Total execution time: 63.9 seconds.

# print number of rhats above 1.1
s <- fit_chains_lower$summary()
s$rhat <- as.numeric(s$rhat)

subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))[, c("variable", "rhat", "ess_bulk", "ess_tail")]

## # A tibble: 0 x 4
## # i 4 variables: variable <chr>, rhat <dbl>, ess_bulk <dbl>, ess_tail <dbl>

# print string if no rhats above 1.1
if (nrow(subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))) == 0) {
  cat("No R-hat values above 1.1\n")
}

## No R-hat values above 1.1

fit_chains_higher <- mod$sample(
  data = stan_data,
  seed = 1234,
  chains = 6,
  parallel_chains = 4,
  threads_per_chain = 1,
  iter_warmup = 1000,
  iter_sampling = 4000,
  refresh = 200,
  adapt_delta = 0.99
)

## Running MCMC with 6 chains, at most 4 in parallel, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 1 Iteration:  200 / 5000 [ 4%] (Warmup)

```



```

## Chain 4 Iteration: 200 / 5000 [ 4%] (Warmup)
## Chain 2 Iteration: 200 / 5000 [ 4%] (Warmup)
## Chain 3 Iteration: 200 / 5000 [ 4%] (Warmup)
## Chain 1 Iteration: 400 / 5000 [ 8%] (Warmup)
## Chain 4 Iteration: 400 / 5000 [ 8%] (Warmup)
## Chain 2 Iteration: 400 / 5000 [ 8%] (Warmup)
## Chain 3 Iteration: 400 / 5000 [ 8%] (Warmup)
## Chain 4 Iteration: 600 / 5000 [12%] (Warmup)
## Chain 1 Iteration: 600 / 5000 [12%] (Warmup)
## Chain 4 Iteration: 800 / 5000 [16%] (Warmup)
## Chain 3 Iteration: 600 / 5000 [12%] (Warmup)
## Chain 1 Iteration: 800 / 5000 [16%] (Warmup)
## Chain 2 Iteration: 600 / 5000 [12%] (Warmup)
## Chain 2 Iteration: 800 / 5000 [16%] (Warmup)
## Chain 3 Iteration: 800 / 5000 [16%] (Warmup)
## Chain 1 Iteration: 1000 / 5000 [20%] (Warmup)
## Chain 1 Iteration: 1001 / 5000 [20%] (Sampling)
## Chain 4 Iteration: 1000 / 5000 [20%] (Warmup)
## Chain 4 Iteration: 1001 / 5000 [20%] (Sampling)
## Chain 2 Iteration: 1000 / 5000 [20%] (Warmup)
## Chain 2 Iteration: 1001 / 5000 [20%] (Sampling)
## Chain 3 Iteration: 1000 / 5000 [20%] (Warmup)
## Chain 3 Iteration: 1001 / 5000 [20%] (Sampling)
## Chain 1 Iteration: 1200 / 5000 [24%] (Sampling)
## Chain 2 Iteration: 1200 / 5000 [24%] (Sampling)
## Chain 4 Iteration: 1200 / 5000 [24%] (Sampling)
## Chain 3 Iteration: 1200 / 5000 [24%] (Sampling)
## Chain 1 Iteration: 1400 / 5000 [28%] (Sampling)
## Chain 2 Iteration: 1400 / 5000 [28%] (Sampling)
## Chain 3 Iteration: 1400 / 5000 [28%] (Sampling)
## Chain 4 Iteration: 1400 / 5000 [28%] (Sampling)
## Chain 2 Iteration: 1600 / 5000 [32%] (Sampling)
## Chain 1 Iteration: 1600 / 5000 [32%] (Sampling)
## Chain 3 Iteration: 1600 / 5000 [32%] (Sampling)
## Chain 2 Iteration: 1800 / 5000 [36%] (Sampling)
## Chain 1 Iteration: 1800 / 5000 [36%] (Sampling)
## Chain 4 Iteration: 1600 / 5000 [32%] (Sampling)
## Chain 2 Iteration: 2000 / 5000 [40%] (Sampling)
## Chain 3 Iteration: 1800 / 5000 [36%] (Sampling)
## Chain 1 Iteration: 2000 / 5000 [40%] (Sampling)
## Chain 2 Iteration: 2200 / 5000 [44%] (Sampling)
## Chain 3 Iteration: 2000 / 5000 [40%] (Sampling)
## Chain 4 Iteration: 1800 / 5000 [36%] (Sampling)
## Chain 1 Iteration: 2200 / 5000 [44%] (Sampling)
## Chain 2 Iteration: 2400 / 5000 [48%] (Sampling)
## Chain 3 Iteration: 2200 / 5000 [44%] (Sampling)
## Chain 2 Iteration: 2600 / 5000 [52%] (Sampling)
## Chain 4 Iteration: 2000 / 5000 [40%] (Sampling)
## Chain 1 Iteration: 2400 / 5000 [48%] (Sampling)
## Chain 3 Iteration: 2400 / 5000 [48%] (Sampling)
## Chain 2 Iteration: 2800 / 5000 [56%] (Sampling)
## Chain 1 Iteration: 2600 / 5000 [52%] (Sampling)
## Chain 4 Iteration: 2200 / 5000 [44%] (Sampling)
## Chain 2 Iteration: 3000 / 5000 [60%] (Sampling)

```

```

## Chain 3 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 1 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 2 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 3 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 4 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 2 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 1 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 3 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 2 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 4 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 1 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 3 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 2 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 1 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 3 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 2 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 4 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 2 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 3 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 1 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 2 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 4 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 3 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 1 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 2 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 3 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 1 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 4 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 2 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 3 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 2 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 2 finished in 55.5 seconds.
## Chain 1 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 5 Iteration: 1 / 5000 [ 0%] (Warmup)
## Chain 4 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 3 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 1 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 3 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 1 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 3 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 1 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 4 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 5 Iteration: 200 / 5000 [ 4%] (Warmup)
## Chain 3 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 3 finished in 65.0 seconds.
## Chain 6 Iteration: 1 / 5000 [ 0%] (Warmup)
## Chain 1 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 1 finished in 66.2 seconds.
## Chain 5 Iteration: 400 / 5000 [ 8%] (Warmup)
## Chain 4 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 5 Iteration: 600 / 5000 [ 12%] (Warmup)
## Chain 4 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 5 Iteration: 800 / 5000 [ 16%] (Warmup)

```

```

## Chain 5 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 6 Iteration: 200 / 5000 [ 4%] (Warmup)
## Chain 5 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 4 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 6 Iteration: 400 / 5000 [ 8%] (Warmup)
## Chain 5 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 4 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 6 Iteration: 600 / 5000 [ 12%] (Warmup)
## Chain 5 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 6 Iteration: 800 / 5000 [ 16%] (Warmup)
## Chain 4 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 6 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 5 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 6 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 4 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 4 finished in 84.7 seconds.
## Chain 6 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 5 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 6 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 5 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 6 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 5 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 6 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 5 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 6 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 5 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 6 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 5 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 6 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 5 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 6 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 5 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 6 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 5 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 6 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 5 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 6 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 5 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 6 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 5 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 6 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 5 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 6 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 5 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 6 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 5 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 6 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 5 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 6 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 6 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 5 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 5 finished in 75.2 seconds.
## Chain 6 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 6 Iteration: 5000 / 5000 [100%] (Sampling)

```

```
## Chain 6 finished in 70.9 seconds.
##
## All 6 chains finished successfully.
## Mean chain execution time: 69.6 seconds.
## Total execution time: 136.1 seconds.

# print number of rhats above 1.1
s <- fit_chains_higher$summary()
s$rhat <- as.numeric(s$rhat)

subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))[, c("variable", "rhat", "ess_bulk", "ess_tail")]

## # A tibble: 14 x 4
##   variable      rhat ess_bulk ess_tail
##   <chr>      <dbl>   <dbl>   <dbl>
## 1 pi[1]      1.34    13.7    69.9
## 2 pi[2]      1.34    13.7    69.9
## 3 theta[1,1,1] 1.61     9.88    67.2
## 4 theta[2,1,1] 1.61     9.87    67.6
## 5 theta[3,1,1] 1.61     9.83    66.0
## 6 theta[1,2,1] 1.61     9.87    65.7
## 7 theta[2,2,1] 1.61     9.87    67.6
## 8 theta[3,2,1] 1.61     9.87    68.8
## 9 theta[1,1,2] 1.61     9.88    67.2
## 10 theta[2,1,2] 1.61     9.87    67.6
## 11 theta[3,1,2] 1.61     9.83    66.0
## 12 theta[1,2,2] 1.61     9.87    65.7
## 13 theta[2,2,2] 1.61     9.87    67.6
## 14 theta[3,2,2] 1.61     9.87    68.8

# print string if no rhats above 1.1
if (nrow(subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))) == 0) {
  cat("No R-hat values above 1.1\n")
}

fit_parallel_chains_lower <- mod$sample(
  data = stan_data,
  seed = 1234,
  chains = 4,
  parallel_chains = 2,
  threads_per_chain = 1,
  iter_warmup = 1000,
  iter_sampling = 4000,
  refresh = 200,
  adapt_delta = 0.99
)

## Running MCMC with 4 chains, at most 2 in parallel, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 1 Iteration:   200 / 5000 [ 4%] (Warmup)
## Chain 2 Iteration:   200 / 5000 [ 4%] (Warmup)
## Chain 1 Iteration:   400 / 5000 [ 8%] (Warmup)
## Chain 2 Iteration:   400 / 5000 [ 8%] (Warmup)
## Chain 1 Iteration:   600 / 5000 [12%] (Warmup)
```

```

## Chain 2 Iteration: 600 / 5000 [ 12%] (Warmup)
## Chain 1 Iteration: 800 / 5000 [ 16%] (Warmup)
## Chain 2 Iteration: 800 / 5000 [ 16%] (Warmup)
## Chain 1 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 1 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 2 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 2 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 1 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 2 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 1 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 2 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 2 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 1 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 2 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 1 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 2 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 1 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 2 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 2 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 1 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 2 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 1 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 2 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 1 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 2 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 2 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 1 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 2 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 1 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 2 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 1 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 2 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 2 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 1 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 2 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 1 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 2 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 1 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 2 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 1 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 2 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 2 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 2 finished in 53.1 seconds.
## Chain 3 Iteration: 1 / 5000 [ 0%] (Warmup)
## Chain 1 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 1 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 1 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 3 Iteration: 200 / 5000 [ 4%] (Warmup)
## Chain 1 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 3 Iteration: 400 / 5000 [ 8%] (Warmup)
## Chain 1 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 1 finished in 64.1 seconds.
## Chain 4 Iteration: 1 / 5000 [ 0%] (Warmup)
## Chain 3 Iteration: 600 / 5000 [ 12%] (Warmup)

```

```

## Chain 3 Iteration: 800 / 5000 [ 16%] (Warmup)
## Chain 3 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 3 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 4 Iteration: 200 / 5000 [ 4%] (Warmup)
## Chain 3 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 4 Iteration: 400 / 5000 [ 8%] (Warmup)
## Chain 3 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 4 Iteration: 600 / 5000 [ 12%] (Warmup)
## Chain 4 Iteration: 800 / 5000 [ 16%] (Warmup)
## Chain 3 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 3 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 4 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 4 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 3 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 4 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 3 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 4 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 3 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 3 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 4 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 3 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 4 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 3 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 3 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 4 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 3 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 4 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 3 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 3 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 4 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 3 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 4 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 3 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 3 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 4 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 3 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 3 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 4 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 3 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 3 finished in 62.5 seconds.
## Chain 4 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 4 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 4 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 4 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 4 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 4 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 4 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 4 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 4 finished in 81.7 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 65.3 seconds.

```

```

## Total execution time: 146.0 seconds.
# print number of rhats above 1.1
s <- fit_parallel_chains_lower$summary()
s$rhat <- as.numeric(s$rhat)

subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))[, c("variable", "rhat", "ess_bulk", "ess_tail")]

## # A tibble: 0 x 4
## # i 4 variables: variable <chr>, rhat <dbl>, ess_bulk <dbl>, ess_tail <dbl>
# print string if no rhats above 1.1
if (nrow(subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))) == 0) {
  cat("No R-hat values above 1.1\n")
}

## No R-hat values above 1.1
fit_parallel_chains_higher <- mod$sample(
  data = stan_data,
  seed = 1234,
  chains = 4,
  parallel_chains = 4,
  threads_per_chain = 1,
  iter_warmup = 1000,
  iter_sampling = 4000,
  refresh = 200,
  adapt_delta = 0.99
)

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 1 Iteration:  200 / 5000 [ 4%] (Warmup)
## Chain 4 Iteration:  200 / 5000 [ 4%] (Warmup)
## Chain 2 Iteration:  200 / 5000 [ 4%] (Warmup)
## Chain 3 Iteration:  200 / 5000 [ 4%] (Warmup)
## Chain 1 Iteration:  400 / 5000 [ 8%] (Warmup)
## Chain 4 Iteration:  400 / 5000 [ 8%] (Warmup)
## Chain 2 Iteration:  400 / 5000 [ 8%] (Warmup)
## Chain 3 Iteration:  400 / 5000 [ 8%] (Warmup)
## Chain 4 Iteration:  600 / 5000 [12%] (Warmup)
## Chain 1 Iteration:  600 / 5000 [12%] (Warmup)
## Chain 4 Iteration:  800 / 5000 [16%] (Warmup)
## Chain 1 Iteration:  800 / 5000 [16%] (Warmup)
## Chain 2 Iteration:  600 / 5000 [12%] (Warmup)
## Chain 3 Iteration:  600 / 5000 [12%] (Warmup)
## Chain 2 Iteration:  800 / 5000 [16%] (Warmup)
## Chain 1 Iteration: 1000 / 5000 [20%] (Warmup)
## Chain 1 Iteration: 1001 / 5000 [20%] (Sampling)
## Chain 3 Iteration:  800 / 5000 [16%] (Warmup)
## Chain 4 Iteration: 1000 / 5000 [20%] (Warmup)
## Chain 4 Iteration: 1001 / 5000 [20%] (Sampling)

```

```

## Chain 2 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 2 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 1 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 3 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 3 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 2 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 4 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 1 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 3 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 2 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 4 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 2 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 1 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 3 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 2 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 3 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 1 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 4 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 2 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 3 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 1 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 2 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 4 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 3 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 1 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 2 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 2 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 3 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 1 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 4 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 2 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 3 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 1 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 4 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 2 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 3 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 1 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 2 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 4 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 3 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 2 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 1 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 2 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 3 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 1 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 4 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 2 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 3 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 1 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 2 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 4 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 3 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 2 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 1 Iteration: 3600 / 5000 [ 72%] (Sampling)

```



```

## Chain 3 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 2 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 4 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 1 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 2 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 3 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 1 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 4 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 2 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 3 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 1 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 2 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 2 finished in 55.8 seconds.
## Chain 3 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 4 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 1 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 3 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 4 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 1 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 3 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 1 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 4 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 3 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 1 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 1 finished in 66.2 seconds.
## Chain 3 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 3 finished in 66.7 seconds.
## Chain 4 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 4 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 4 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 4 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 4 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 4 finished in 84.2 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 68.2 seconds.
## Total execution time: 84.3 seconds.

# print number of rhats above 1.1
s <- fit_paral_chains_higher$summary()
s$rhat <- as.numeric(s$rhat)

subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))[, c("variable", "rhat", "ess_bulk", "ess_tail")]

## # A tibble: 0 x 4
## # i 4 variables: variable <chr>, rhat <dbl>, ess_bulk <dbl>, ess_tail <dbl>

# print string if no rhats above 1.1
if (nrow(subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))) == 0) {
  cat("No R-hat values above 1.1\n")
}

## No R-hat values above 1.1

```

```

fit_warmup_lower <- mod$sample(
  data = stan_data,
  seed = 1234,
  chains = 4,
  parallel_chains = 4,
  threads_per_chain = 1,
  iter_warmup = 500,
  iter_sampling = 4000,
  refresh = 200,
  adapt_delta = 0.99
)

```

Running MCMC with 4 parallel chains, with 1 thread(s) per chain...

```

##
## Chain 1 Iteration:    1 / 4500 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 4500 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 4500 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 4500 [ 0%] (Warmup)
## Chain 1 Iteration:   200 / 4500 [ 4%] (Warmup)
## Chain 4 Iteration:   200 / 4500 [ 4%] (Warmup)
## Chain 2 Iteration:   200 / 4500 [ 4%] (Warmup)
## Chain 3 Iteration:   200 / 4500 [ 4%] (Warmup)
## Chain 1 Iteration:   400 / 4500 [ 8%] (Warmup)
## Chain 4 Iteration:   400 / 4500 [ 8%] (Warmup)
## Chain 4 Iteration:   501 / 4500 [ 11%] (Sampling)
## Chain 2 Iteration:   400 / 4500 [ 8%] (Warmup)
## Chain 3 Iteration:   400 / 4500 [ 8%] (Warmup)
## Chain 1 Iteration:   501 / 4500 [ 11%] (Sampling)
## Chain 3 Iteration:   501 / 4500 [ 11%] (Sampling)
## Chain 4 Iteration:   700 / 4500 [ 15%] (Sampling)
## Chain 2 Iteration:   501 / 4500 [ 11%] (Sampling)
## Chain 1 Iteration:   700 / 4500 [ 15%] (Sampling)
## Chain 4 Iteration:   900 / 4500 [ 20%] (Sampling)
## Chain 3 Iteration:   700 / 4500 [ 15%] (Sampling)
## Chain 2 Iteration:   700 / 4500 [ 15%] (Sampling)
## Chain 4 Iteration:  1100 / 4500 [ 24%] (Sampling)
## Chain 1 Iteration:   900 / 4500 [ 20%] (Sampling)
## Chain 3 Iteration:   900 / 4500 [ 20%] (Sampling)
## Chain 4 Iteration:  1300 / 4500 [ 28%] (Sampling)
## Chain 2 Iteration:   900 / 4500 [ 20%] (Sampling)
## Chain 4 Iteration:  1500 / 4500 [ 33%] (Sampling)
## Chain 1 Iteration:  1100 / 4500 [ 24%] (Sampling)
## Chain 3 Iteration:  1100 / 4500 [ 24%] (Sampling)
## Chain 4 Iteration:  1700 / 4500 [ 37%] (Sampling)
## Chain 2 Iteration:  1100 / 4500 [ 24%] (Sampling)
## Chain 4 Iteration:  1900 / 4500 [ 42%] (Sampling)
## Chain 3 Iteration:  1300 / 4500 [ 28%] (Sampling)
## Chain 1 Iteration:  1300 / 4500 [ 28%] (Sampling)
## Chain 2 Iteration:  1300 / 4500 [ 28%] (Sampling)
## Chain 4 Iteration:  2100 / 4500 [ 46%] (Sampling)
## Chain 3 Iteration:  1500 / 4500 [ 33%] (Sampling)
## Chain 1 Iteration:  1500 / 4500 [ 33%] (Sampling)
## Chain 4 Iteration:  2300 / 4500 [ 51%] (Sampling)
## Chain 2 Iteration:  1500 / 4500 [ 33%] (Sampling)

```

```

## Chain 4 Iteration: 2500 / 4500 [ 55%] (Sampling)
## Chain 3 Iteration: 1700 / 4500 [ 37%] (Sampling)
## Chain 1 Iteration: 1700 / 4500 [ 37%] (Sampling)
## Chain 4 Iteration: 2700 / 4500 [ 60%] (Sampling)
## Chain 2 Iteration: 1700 / 4500 [ 37%] (Sampling)
## Chain 3 Iteration: 1900 / 4500 [ 42%] (Sampling)
## Chain 4 Iteration: 2900 / 4500 [ 64%] (Sampling)
## Chain 1 Iteration: 1900 / 4500 [ 42%] (Sampling)
## Chain 2 Iteration: 1900 / 4500 [ 42%] (Sampling)
## Chain 4 Iteration: 3100 / 4500 [ 68%] (Sampling)
## Chain 3 Iteration: 2100 / 4500 [ 46%] (Sampling)
## Chain 4 Iteration: 3300 / 4500 [ 73%] (Sampling)
## Chain 1 Iteration: 2100 / 4500 [ 46%] (Sampling)
## Chain 2 Iteration: 2100 / 4500 [ 46%] (Sampling)
## Chain 3 Iteration: 2300 / 4500 [ 51%] (Sampling)
## Chain 4 Iteration: 3500 / 4500 [ 77%] (Sampling)
## Chain 1 Iteration: 2300 / 4500 [ 51%] (Sampling)
## Chain 4 Iteration: 3700 / 4500 [ 82%] (Sampling)
## Chain 3 Iteration: 2500 / 4500 [ 55%] (Sampling)
## Chain 2 Iteration: 2300 / 4500 [ 51%] (Sampling)
## Chain 4 Iteration: 3900 / 4500 [ 86%] (Sampling)
## Chain 1 Iteration: 2500 / 4500 [ 55%] (Sampling)
## Chain 3 Iteration: 2700 / 4500 [ 60%] (Sampling)
## Chain 2 Iteration: 2500 / 4500 [ 55%] (Sampling)
## Chain 4 Iteration: 4100 / 4500 [ 91%] (Sampling)
## Chain 4 Iteration: 4300 / 4500 [ 95%] (Sampling)
## Chain 1 Iteration: 2700 / 4500 [ 60%] (Sampling)
## Chain 3 Iteration: 2900 / 4500 [ 64%] (Sampling)
## Chain 2 Iteration: 2700 / 4500 [ 60%] (Sampling)
## Chain 4 Iteration: 4500 / 4500 [100%] (Sampling)
## Chain 4 finished in 48.6 seconds.
## Chain 1 Iteration: 2900 / 4500 [ 64%] (Sampling)
## Chain 3 Iteration: 3100 / 4500 [ 68%] (Sampling)
## Chain 2 Iteration: 2900 / 4500 [ 64%] (Sampling)
## Chain 3 Iteration: 3300 / 4500 [ 73%] (Sampling)
## Chain 1 Iteration: 3100 / 4500 [ 68%] (Sampling)
## Chain 2 Iteration: 3100 / 4500 [ 68%] (Sampling)
## Chain 3 Iteration: 3500 / 4500 [ 77%] (Sampling)
## Chain 1 Iteration: 3300 / 4500 [ 73%] (Sampling)
## Chain 2 Iteration: 3300 / 4500 [ 73%] (Sampling)
## Chain 3 Iteration: 3700 / 4500 [ 82%] (Sampling)
## Chain 1 Iteration: 3500 / 4500 [ 77%] (Sampling)
## Chain 2 Iteration: 3500 / 4500 [ 77%] (Sampling)
## Chain 3 Iteration: 3900 / 4500 [ 86%] (Sampling)
## Chain 1 Iteration: 3700 / 4500 [ 82%] (Sampling)
## Chain 2 Iteration: 3700 / 4500 [ 82%] (Sampling)
## Chain 3 Iteration: 4100 / 4500 [ 91%] (Sampling)
## Chain 1 Iteration: 3900 / 4500 [ 86%] (Sampling)
## Chain 2 Iteration: 3900 / 4500 [ 86%] (Sampling)
## Chain 3 Iteration: 4300 / 4500 [ 95%] (Sampling)
## Chain 1 Iteration: 4100 / 4500 [ 91%] (Sampling)
## Chain 2 Iteration: 4100 / 4500 [ 91%] (Sampling)
## Chain 3 Iteration: 4500 / 4500 [100%] (Sampling)
## Chain 3 finished in 71.6 seconds.

```

```

## Chain 2 Iteration: 4300 / 4500 [ 95%] (Sampling)
## Chain 1 Iteration: 4300 / 4500 [ 95%] (Sampling)
## Chain 2 Iteration: 4500 / 4500 [100%] (Sampling)
## Chain 2 finished in 77.6 seconds.
## Chain 1 Iteration: 4500 / 4500 [100%] (Sampling)
## Chain 1 finished in 77.8 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 68.9 seconds.
## Total execution time: 77.9 seconds.

# print number of rhats above 1.1
s <- fit_warmup_lower$summary()
s$rhat <- as.numeric(s$rhat)

subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))[, c("variable", "rhat", "ess_bulk", "ess_tail")]

## # A tibble: 0 x 4
## # i 4 variables: variable <chr>, rhat <dbl>, ess_bulk <dbl>, ess_tail <dbl>

# print string if no rhats above 1.1
if (nrow(subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))) == 0) {
  cat("No R-hat values above 1.1\n")
}

## No R-hat values above 1.1

fit_warmup_higher <- mod$sample(
  data = stan_data,
  seed = 1234,
  chains = 4,
  parallel_chains = 4,
  threads_per_chain = 1,
  iter_warmup = 2000,
  iter_sampling = 4000,
  refresh = 200,
  adapt_delta = 0.99
)

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 1 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 4 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 2 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 3 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 1 Iteration:  400 / 6000 [ 6%] (Warmup)
## Chain 4 Iteration:  400 / 6000 [ 6%] (Warmup)
## Chain 3 Iteration:  400 / 6000 [ 6%] (Warmup)
## Chain 2 Iteration:  400 / 6000 [ 6%] (Warmup)
## Chain 4 Iteration:  600 / 6000 [10%] (Warmup)
## Chain 1 Iteration:  600 / 6000 [10%] (Warmup)
## Chain 4 Iteration:  800 / 6000 [13%] (Warmup)

```

```

## Chain 3 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 2 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 1 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 3 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 2 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 4 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 1 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 3 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 4 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 1 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 2 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 3 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 4 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 1 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 2 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 1 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 2 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 3 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 4 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 3 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 1 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 2 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 4 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 1 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 2 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 3 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 4 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 1 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 4 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 2 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 2 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 1 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 3 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 3 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 4 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 1 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 2 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 3 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 1 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 4 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 2 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 1 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 3 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 4 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 1 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 2 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 3 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 4 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 1 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 2 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 1 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 4 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 3 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 2 Iteration: 3000 / 6000 [ 50%] (Sampling)

```

```

## Chain 1 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 4 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 1 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 3 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 2 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 1 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 4 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 2 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 3 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 1 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 4 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 1 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 2 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 3 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 1 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 4 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 2 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 3 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 1 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 2 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 4 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 1 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 3 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 1 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 2 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 4 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 1 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 3 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 2 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 4 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 1 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 2 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 3 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 1 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 4 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 1 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 1 finished in 63.5 seconds.
## Chain 2 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 3 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 4 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 2 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 3 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 4 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 2 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 3 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 4 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 2 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 4 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 3 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 2 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 4 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 3 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 2 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 4 Iteration: 5800 / 6000 [ 96%] (Sampling)

```

```

## Chain 3 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 2 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 2 finished in 80.5 seconds.
## Chain 4 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 4 finished in 81.6 seconds.
## Chain 3 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 3 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 3 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 3 finished in 88.9 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 78.6 seconds.
## Total execution time: 88.9 seconds.

# print number of rhats above 1.1
s <- fit_warmup_higher$summary()
s$rhat <- as.numeric(s$rhat)

subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))[, c("variable", "rhat", "ess_bulk", "ess_tail")]

## # A tibble: 0 x 4
## # i 4 variables: variable <chr>, rhat <dbl>, ess_bulk <dbl>, ess_tail <dbl>

# print string if no rhats above 1.1
if (nrow(subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))) == 0) {
  cat("No R-hat values above 1.1\n")
}

## No R-hat values above 1.1

fit_sampling_lower <- mod$sample(
  data = stan_data,
  seed = 1234,
  chains = 4,
  parallel_chains = 4,
  threads_per_chain = 1,
  iter_warmup = 1000,
  iter_sampling = 2000,
  refresh = 200,
  adapt_delta = 0.99
)

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 3000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 3000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 3000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 3000 [ 0%] (Warmup)
## Chain 1 Iteration:  200 / 3000 [ 6%] (Warmup)
## Chain 4 Iteration:  200 / 3000 [ 6%] (Warmup)
## Chain 2 Iteration:  200 / 3000 [ 6%] (Warmup)
## Chain 3 Iteration:  200 / 3000 [ 6%] (Warmup)
## Chain 1 Iteration:  400 / 3000 [13%] (Warmup)
## Chain 4 Iteration:  400 / 3000 [13%] (Warmup)
## Chain 2 Iteration:  400 / 3000 [13%] (Warmup)
## Chain 3 Iteration:  400 / 3000 [13%] (Warmup)

```

```

## Chain 4 Iteration: 600 / 3000 [ 20%] (Warmup)
## Chain 1 Iteration: 600 / 3000 [ 20%] (Warmup)
## Chain 2 Iteration: 600 / 3000 [ 20%] (Warmup)
## Chain 4 Iteration: 800 / 3000 [ 26%] (Warmup)
## Chain 1 Iteration: 800 / 3000 [ 26%] (Warmup)
## Chain 3 Iteration: 600 / 3000 [ 20%] (Warmup)
## Chain 2 Iteration: 800 / 3000 [ 26%] (Warmup)
## Chain 3 Iteration: 800 / 3000 [ 26%] (Warmup)
## Chain 1 Iteration: 1000 / 3000 [ 33%] (Warmup)
## Chain 1 Iteration: 1001 / 3000 [ 33%] (Sampling)
## Chain 4 Iteration: 1000 / 3000 [ 33%] (Warmup)
## Chain 4 Iteration: 1001 / 3000 [ 33%] (Sampling)
## Chain 2 Iteration: 1000 / 3000 [ 33%] (Warmup)
## Chain 2 Iteration: 1001 / 3000 [ 33%] (Sampling)
## Chain 3 Iteration: 1000 / 3000 [ 33%] (Warmup)
## Chain 3 Iteration: 1001 / 3000 [ 33%] (Sampling)
## Chain 1 Iteration: 1200 / 3000 [ 40%] (Sampling)
## Chain 2 Iteration: 1200 / 3000 [ 40%] (Sampling)
## Chain 4 Iteration: 1200 / 3000 [ 40%] (Sampling)
## Chain 3 Iteration: 1200 / 3000 [ 40%] (Sampling)
## Chain 1 Iteration: 1400 / 3000 [ 46%] (Sampling)
## Chain 2 Iteration: 1400 / 3000 [ 46%] (Sampling)
## Chain 3 Iteration: 1400 / 3000 [ 46%] (Sampling)
## Chain 2 Iteration: 1600 / 3000 [ 53%] (Sampling)
## Chain 4 Iteration: 1400 / 3000 [ 46%] (Sampling)
## Chain 1 Iteration: 1600 / 3000 [ 53%] (Sampling)
## Chain 2 Iteration: 1800 / 3000 [ 60%] (Sampling)
## Chain 3 Iteration: 1600 / 3000 [ 53%] (Sampling)
## Chain 1 Iteration: 1800 / 3000 [ 60%] (Sampling)
## Chain 4 Iteration: 1600 / 3000 [ 53%] (Sampling)
## Chain 2 Iteration: 2000 / 3000 [ 66%] (Sampling)
## Chain 3 Iteration: 1800 / 3000 [ 60%] (Sampling)
## Chain 1 Iteration: 2000 / 3000 [ 66%] (Sampling)
## Chain 2 Iteration: 2200 / 3000 [ 73%] (Sampling)
## Chain 3 Iteration: 2000 / 3000 [ 66%] (Sampling)
## Chain 4 Iteration: 1800 / 3000 [ 60%] (Sampling)
## Chain 2 Iteration: 2400 / 3000 [ 80%] (Sampling)
## Chain 1 Iteration: 2200 / 3000 [ 73%] (Sampling)
## Chain 3 Iteration: 2200 / 3000 [ 73%] (Sampling)
## Chain 2 Iteration: 2600 / 3000 [ 86%] (Sampling)
## Chain 1 Iteration: 2400 / 3000 [ 80%] (Sampling)
## Chain 4 Iteration: 2000 / 3000 [ 66%] (Sampling)
## Chain 2 Iteration: 2800 / 3000 [ 93%] (Sampling)
## Chain 3 Iteration: 2400 / 3000 [ 80%] (Sampling)
## Chain 1 Iteration: 2600 / 3000 [ 86%] (Sampling)
## Chain 2 Iteration: 3000 / 3000 [100%] (Sampling)
## Chain 2 finished in 35.1 seconds.
## Chain 3 Iteration: 2600 / 3000 [ 86%] (Sampling)
## Chain 4 Iteration: 2200 / 3000 [ 73%] (Sampling)
## Chain 1 Iteration: 2800 / 3000 [ 93%] (Sampling)
## Chain 3 Iteration: 2800 / 3000 [ 93%] (Sampling)
## Chain 4 Iteration: 2400 / 3000 [ 80%] (Sampling)
## Chain 1 Iteration: 3000 / 3000 [100%] (Sampling)
## Chain 1 finished in 40.0 seconds.

```



```

## Chain 3 Iteration: 3000 / 3000 [100%] (Sampling)
## Chain 3 finished in 40.5 seconds.
## Chain 4 Iteration: 2600 / 3000 [ 86%] (Sampling)
## Chain 4 Iteration: 2800 / 3000 [ 93%] (Sampling)
## Chain 4 Iteration: 3000 / 3000 [100%] (Sampling)
## Chain 4 finished in 49.9 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 41.4 seconds.
## Total execution time: 50.1 seconds.

# print number of rhats above 1.1
s <- fit_sampling_lower$summary()
s$rhat <- as.numeric(s$rhat)

subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))[, c("variable", "rhat", "ess_bulk", "ess_tail")]

## # A tibble: 0 x 4
## # i 4 variables: variable <chr>, rhat <dbl>, ess_bulk <dbl>, ess_tail <dbl>

# print string if no rhats above 1.1
if (nrow(subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))) == 0) {
  cat("No R-hat values above 1.1\n")
}

## No R-hat values above 1.1

fit_sampling_higher <- mod$sample(
  data = stan_data,
  seed = 1234,
  chains = 4,
  parallel_chains = 4,
  threads_per_chain = 1,
  iter_warmup = 1000,
  iter_sampling = 8000,
  refresh = 200,
  adapt_delta = 0.99
)

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 9000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 9000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 9000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 9000 [ 0%] (Warmup)
## Chain 1 Iteration:  200 / 9000 [ 2%] (Warmup)
## Chain 4 Iteration:  200 / 9000 [ 2%] (Warmup)
## Chain 2 Iteration:  200 / 9000 [ 2%] (Warmup)
## Chain 3 Iteration:  200 / 9000 [ 2%] (Warmup)
## Chain 1 Iteration:  400 / 9000 [ 4%] (Warmup)
## Chain 4 Iteration:  400 / 9000 [ 4%] (Warmup)
## Chain 3 Iteration:  400 / 9000 [ 4%] (Warmup)
## Chain 2 Iteration:  400 / 9000 [ 4%] (Warmup)
## Chain 4 Iteration:  600 / 9000 [ 6%] (Warmup)
## Chain 1 Iteration:  600 / 9000 [ 6%] (Warmup)
## Chain 4 Iteration:  800 / 9000 [ 8%] (Warmup)

```

```

## Chain 2 Iteration: 600 / 9000 [ 6%] (Warmup)
## Chain 3 Iteration: 600 / 9000 [ 6%] (Warmup)
## Chain 1 Iteration: 800 / 9000 [ 8%] (Warmup)
## Chain 2 Iteration: 800 / 9000 [ 8%] (Warmup)
## Chain 3 Iteration: 800 / 9000 [ 8%] (Warmup)
## Chain 1 Iteration: 1000 / 9000 [ 11%] (Warmup)
## Chain 1 Iteration: 1001 / 9000 [ 11%] (Sampling)
## Chain 4 Iteration: 1000 / 9000 [ 11%] (Warmup)
## Chain 4 Iteration: 1001 / 9000 [ 11%] (Sampling)
## Chain 2 Iteration: 1000 / 9000 [ 11%] (Warmup)
## Chain 2 Iteration: 1001 / 9000 [ 11%] (Sampling)
## Chain 3 Iteration: 1000 / 9000 [ 11%] (Warmup)
## Chain 3 Iteration: 1001 / 9000 [ 11%] (Sampling)
## Chain 1 Iteration: 1200 / 9000 [ 13%] (Sampling)
## Chain 2 Iteration: 1200 / 9000 [ 13%] (Sampling)
## Chain 4 Iteration: 1200 / 9000 [ 13%] (Sampling)
## Chain 3 Iteration: 1200 / 9000 [ 13%] (Sampling)
## Chain 1 Iteration: 1400 / 9000 [ 15%] (Sampling)
## Chain 2 Iteration: 1400 / 9000 [ 15%] (Sampling)
## Chain 3 Iteration: 1400 / 9000 [ 15%] (Sampling)
## Chain 4 Iteration: 1400 / 9000 [ 15%] (Sampling)
## Chain 2 Iteration: 1600 / 9000 [ 17%] (Sampling)
## Chain 1 Iteration: 1600 / 9000 [ 17%] (Sampling)
## Chain 3 Iteration: 1600 / 9000 [ 17%] (Sampling)
## Chain 2 Iteration: 1800 / 9000 [ 20%] (Sampling)
## Chain 1 Iteration: 1800 / 9000 [ 20%] (Sampling)
## Chain 4 Iteration: 1600 / 9000 [ 17%] (Sampling)
## Chain 2 Iteration: 2000 / 9000 [ 22%] (Sampling)
## Chain 3 Iteration: 1800 / 9000 [ 20%] (Sampling)
## Chain 2 Iteration: 2200 / 9000 [ 24%] (Sampling)
## Chain 1 Iteration: 2000 / 9000 [ 22%] (Sampling)
## Chain 3 Iteration: 2000 / 9000 [ 22%] (Sampling)
## Chain 4 Iteration: 1800 / 9000 [ 20%] (Sampling)
## Chain 2 Iteration: 2400 / 9000 [ 26%] (Sampling)
## Chain 1 Iteration: 2200 / 9000 [ 24%] (Sampling)
## Chain 3 Iteration: 2200 / 9000 [ 24%] (Sampling)
## Chain 2 Iteration: 2600 / 9000 [ 28%] (Sampling)
## Chain 4 Iteration: 2000 / 9000 [ 22%] (Sampling)
## Chain 1 Iteration: 2400 / 9000 [ 26%] (Sampling)
## Chain 3 Iteration: 2400 / 9000 [ 26%] (Sampling)
## Chain 2 Iteration: 2800 / 9000 [ 31%] (Sampling)
## Chain 1 Iteration: 2600 / 9000 [ 28%] (Sampling)
## Chain 2 Iteration: 3000 / 9000 [ 33%] (Sampling)
## Chain 4 Iteration: 2200 / 9000 [ 24%] (Sampling)
## Chain 3 Iteration: 2600 / 9000 [ 28%] (Sampling)
## Chain 2 Iteration: 3200 / 9000 [ 35%] (Sampling)
## Chain 3 Iteration: 2800 / 9000 [ 31%] (Sampling)
## Chain 1 Iteration: 2800 / 9000 [ 31%] (Sampling)
## Chain 4 Iteration: 2400 / 9000 [ 26%] (Sampling)
## Chain 2 Iteration: 3400 / 9000 [ 37%] (Sampling)
## Chain 3 Iteration: 3000 / 9000 [ 33%] (Sampling)
## Chain 1 Iteration: 3000 / 9000 [ 33%] (Sampling)
## Chain 2 Iteration: 3600 / 9000 [ 40%] (Sampling)
## Chain 4 Iteration: 2600 / 9000 [ 28%] (Sampling)

```

```

## Chain 3 Iteration: 3200 / 9000 [ 35%] (Sampling)
## Chain 2 Iteration: 3800 / 9000 [ 42%] (Sampling)
## Chain 1 Iteration: 3200 / 9000 [ 35%] (Sampling)
## Chain 3 Iteration: 3400 / 9000 [ 37%] (Sampling)
## Chain 2 Iteration: 4000 / 9000 [ 44%] (Sampling)
## Chain 1 Iteration: 3400 / 9000 [ 37%] (Sampling)
## Chain 4 Iteration: 2800 / 9000 [ 31%] (Sampling)
## Chain 2 Iteration: 4200 / 9000 [ 46%] (Sampling)
## Chain 3 Iteration: 3600 / 9000 [ 40%] (Sampling)
## Chain 1 Iteration: 3600 / 9000 [ 40%] (Sampling)
## Chain 2 Iteration: 4400 / 9000 [ 48%] (Sampling)
## Chain 4 Iteration: 3000 / 9000 [ 33%] (Sampling)
## Chain 3 Iteration: 3800 / 9000 [ 42%] (Sampling)
## Chain 1 Iteration: 3800 / 9000 [ 42%] (Sampling)
## Chain 2 Iteration: 4600 / 9000 [ 51%] (Sampling)
## Chain 3 Iteration: 4000 / 9000 [ 44%] (Sampling)
## Chain 4 Iteration: 3200 / 9000 [ 35%] (Sampling)
## Chain 2 Iteration: 4800 / 9000 [ 53%] (Sampling)
## Chain 1 Iteration: 4000 / 9000 [ 44%] (Sampling)
## Chain 3 Iteration: 4200 / 9000 [ 46%] (Sampling)
## Chain 2 Iteration: 5000 / 9000 [ 55%] (Sampling)
## Chain 4 Iteration: 3400 / 9000 [ 37%] (Sampling)
## Chain 1 Iteration: 4200 / 9000 [ 46%] (Sampling)
## Chain 2 Iteration: 5200 / 9000 [ 57%] (Sampling)
## Chain 3 Iteration: 4400 / 9000 [ 48%] (Sampling)
## Chain 2 Iteration: 5400 / 9000 [ 60%] (Sampling)
## Chain 1 Iteration: 4400 / 9000 [ 48%] (Sampling)
## Chain 3 Iteration: 4600 / 9000 [ 51%] (Sampling)
## Chain 4 Iteration: 3600 / 9000 [ 40%] (Sampling)
## Chain 2 Iteration: 5600 / 9000 [ 62%] (Sampling)
## Chain 1 Iteration: 4600 / 9000 [ 51%] (Sampling)
## Chain 3 Iteration: 4800 / 9000 [ 53%] (Sampling)
## Chain 2 Iteration: 5800 / 9000 [ 64%] (Sampling)
## Chain 4 Iteration: 3800 / 9000 [ 42%] (Sampling)
## Chain 1 Iteration: 4800 / 9000 [ 53%] (Sampling)
## Chain 3 Iteration: 5000 / 9000 [ 55%] (Sampling)
## Chain 2 Iteration: 6000 / 9000 [ 66%] (Sampling)
## Chain 4 Iteration: 4000 / 9000 [ 44%] (Sampling)
## Chain 2 Iteration: 6200 / 9000 [ 68%] (Sampling)
## Chain 1 Iteration: 5000 / 9000 [ 55%] (Sampling)
## Chain 3 Iteration: 5200 / 9000 [ 57%] (Sampling)
## Chain 2 Iteration: 6400 / 9000 [ 71%] (Sampling)
## Chain 3 Iteration: 5400 / 9000 [ 60%] (Sampling)
## Chain 1 Iteration: 5200 / 9000 [ 57%] (Sampling)
## Chain 4 Iteration: 4200 / 9000 [ 46%] (Sampling)
## Chain 2 Iteration: 6600 / 9000 [ 73%] (Sampling)
## Chain 3 Iteration: 5600 / 9000 [ 62%] (Sampling)
## Chain 1 Iteration: 5400 / 9000 [ 60%] (Sampling)
## Chain 2 Iteration: 6800 / 9000 [ 75%] (Sampling)
## Chain 4 Iteration: 4400 / 9000 [ 48%] (Sampling)
## Chain 3 Iteration: 5800 / 9000 [ 64%] (Sampling)
## Chain 2 Iteration: 7000 / 9000 [ 77%] (Sampling)
## Chain 1 Iteration: 5600 / 9000 [ 62%] (Sampling)
## Chain 3 Iteration: 6000 / 9000 [ 66%] (Sampling)

```

```

## Chain 2 Iteration: 7200 / 9000 [ 80%] (Sampling)
## Chain 4 Iteration: 4600 / 9000 [ 51%] (Sampling)
## Chain 1 Iteration: 5800 / 9000 [ 64%] (Sampling)
## Chain 2 Iteration: 7400 / 9000 [ 82%] (Sampling)
## Chain 3 Iteration: 6200 / 9000 [ 68%] (Sampling)
## Chain 1 Iteration: 6000 / 9000 [ 66%] (Sampling)
## Chain 2 Iteration: 7600 / 9000 [ 84%] (Sampling)
## Chain 4 Iteration: 4800 / 9000 [ 53%] (Sampling)
## Chain 3 Iteration: 6400 / 9000 [ 71%] (Sampling)
## Chain 2 Iteration: 7800 / 9000 [ 86%] (Sampling)
## Chain 1 Iteration: 6200 / 9000 [ 68%] (Sampling)
## Chain 3 Iteration: 6600 / 9000 [ 73%] (Sampling)
## Chain 4 Iteration: 5000 / 9000 [ 55%] (Sampling)
## Chain 2 Iteration: 8000 / 9000 [ 88%] (Sampling)
## Chain 1 Iteration: 6400 / 9000 [ 71%] (Sampling)
## Chain 3 Iteration: 6800 / 9000 [ 75%] (Sampling)
## Chain 2 Iteration: 8200 / 9000 [ 91%] (Sampling)
## Chain 4 Iteration: 5200 / 9000 [ 57%] (Sampling)
## Chain 1 Iteration: 6600 / 9000 [ 73%] (Sampling)
## Chain 2 Iteration: 8400 / 9000 [ 93%] (Sampling)
## Chain 3 Iteration: 7000 / 9000 [ 77%] (Sampling)
## Chain 2 Iteration: 8600 / 9000 [ 95%] (Sampling)
## Chain 1 Iteration: 6800 / 9000 [ 75%] (Sampling)
## Chain 4 Iteration: 5400 / 9000 [ 60%] (Sampling)
## Chain 3 Iteration: 7200 / 9000 [ 80%] (Sampling)
## Chain 2 Iteration: 8800 / 9000 [ 97%] (Sampling)
## Chain 1 Iteration: 7000 / 9000 [ 77%] (Sampling)
## Chain 3 Iteration: 7400 / 9000 [ 82%] (Sampling)
## Chain 2 Iteration: 9000 / 9000 [100%] (Sampling)
## Chain 4 Iteration: 5600 / 9000 [ 62%] (Sampling)
## Chain 2 finished in 94.8 seconds.
## Chain 1 Iteration: 7200 / 9000 [ 80%] (Sampling)
## Chain 3 Iteration: 7600 / 9000 [ 84%] (Sampling)
## Chain 4 Iteration: 5800 / 9000 [ 64%] (Sampling)
## Chain 1 Iteration: 7400 / 9000 [ 82%] (Sampling)
## Chain 3 Iteration: 7800 / 9000 [ 86%] (Sampling)
## Chain 3 Iteration: 8000 / 9000 [ 88%] (Sampling)
## Chain 1 Iteration: 7600 / 9000 [ 84%] (Sampling)
## Chain 4 Iteration: 6000 / 9000 [ 66%] (Sampling)
## Chain 3 Iteration: 8200 / 9000 [ 91%] (Sampling)
## Chain 1 Iteration: 7800 / 9000 [ 86%] (Sampling)
## Chain 4 Iteration: 6200 / 9000 [ 68%] (Sampling)
## Chain 3 Iteration: 8400 / 9000 [ 93%] (Sampling)
## Chain 1 Iteration: 8000 / 9000 [ 88%] (Sampling)
## Chain 3 Iteration: 8600 / 9000 [ 95%] (Sampling)
## Chain 4 Iteration: 6400 / 9000 [ 71%] (Sampling)
## Chain 1 Iteration: 8200 / 9000 [ 91%] (Sampling)
## Chain 3 Iteration: 8800 / 9000 [ 97%] (Sampling)
## Chain 4 Iteration: 6600 / 9000 [ 73%] (Sampling)
## Chain 1 Iteration: 8400 / 9000 [ 93%] (Sampling)
## Chain 3 Iteration: 9000 / 9000 [100%] (Sampling)
## Chain 3 finished in 113.6 seconds.
## Chain 1 Iteration: 8600 / 9000 [ 95%] (Sampling)
## Chain 4 Iteration: 6800 / 9000 [ 75%] (Sampling)

```

```

## Chain 1 Iteration: 8800 / 9000 [ 97%] (Sampling)
## Chain 4 Iteration: 7000 / 9000 [ 77%] (Sampling)
## Chain 1 Iteration: 9000 / 9000 [100%] (Sampling)
## Chain 1 finished in 120.1 seconds.
## Chain 4 Iteration: 7200 / 9000 [ 80%] (Sampling)
## Chain 4 Iteration: 7400 / 9000 [ 82%] (Sampling)
## Chain 4 Iteration: 7600 / 9000 [ 84%] (Sampling)
## Chain 4 Iteration: 7800 / 9000 [ 86%] (Sampling)
## Chain 4 Iteration: 8000 / 9000 [ 88%] (Sampling)
## Chain 4 Iteration: 8200 / 9000 [ 91%] (Sampling)
## Chain 4 Iteration: 8400 / 9000 [ 93%] (Sampling)
## Chain 4 Iteration: 8600 / 9000 [ 95%] (Sampling)
## Chain 4 Iteration: 8800 / 9000 [ 97%] (Sampling)
## Chain 4 Iteration: 9000 / 9000 [100%] (Sampling)
## Chain 4 finished in 152.4 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 120.2 seconds.
## Total execution time: 152.5 seconds.

# print number of rhats above 1.1
s <- fit_sampling_higher$summary()
s$rhat <- as.numeric(s$rhat)

subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))[, c("variable", "rhat", "ess_bulk", "ess_tail")]

## # A tibble: 0 x 4
## # i 4 variables: variable <chr>, rhat <dbl>, ess_bulk <dbl>, ess_tail <dbl>

# print string if no rhats above 1.1
if (nrow(subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))) == 0) {
  cat("No R-hat values above 1.1\n")
}

## No R-hat values above 1.1

fit_refresh_lower <- mod$sample(
  data = stan_data,
  seed = 1234,
  chains = 4,
  parallel_chains = 4,
  threads_per_chain = 1,
  iter_warmup = 1000,
  iter_sampling = 4000,
  refresh = 100,
  adapt_delta = 0.99
)

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 2 Iteration:  100 / 5000 [ 2%] (Warmup)
## Chain 1 Iteration:  100 / 5000 [ 2%] (Warmup)

```

```

## Chain 3 Iteration: 100 / 5000 [ 2%] (Warmup)
## Chain 4 Iteration: 100 / 5000 [ 2%] (Warmup)
## Chain 1 Iteration: 200 / 5000 [ 4%] (Warmup)
## Chain 4 Iteration: 200 / 5000 [ 4%] (Warmup)
## Chain 2 Iteration: 200 / 5000 [ 4%] (Warmup)
## Chain 1 Iteration: 300 / 5000 [ 6%] (Warmup)
## Chain 3 Iteration: 200 / 5000 [ 4%] (Warmup)
## Chain 4 Iteration: 300 / 5000 [ 6%] (Warmup)
## Chain 1 Iteration: 400 / 5000 [ 8%] (Warmup)
## Chain 4 Iteration: 400 / 5000 [ 8%] (Warmup)
## Chain 2 Iteration: 300 / 5000 [ 6%] (Warmup)
## Chain 3 Iteration: 300 / 5000 [ 6%] (Warmup)
## Chain 4 Iteration: 500 / 5000 [ 10%] (Warmup)
## Chain 1 Iteration: 500 / 5000 [ 10%] (Warmup)
## Chain 2 Iteration: 400 / 5000 [ 8%] (Warmup)
## Chain 3 Iteration: 400 / 5000 [ 8%] (Warmup)
## Chain 4 Iteration: 600 / 5000 [ 12%] (Warmup)
## Chain 1 Iteration: 600 / 5000 [ 12%] (Warmup)
## Chain 4 Iteration: 700 / 5000 [ 14%] (Warmup)
## Chain 1 Iteration: 700 / 5000 [ 14%] (Warmup)
## Chain 2 Iteration: 500 / 5000 [ 10%] (Warmup)
## Chain 3 Iteration: 500 / 5000 [ 10%] (Warmup)
## Chain 4 Iteration: 800 / 5000 [ 16%] (Warmup)
## Chain 1 Iteration: 800 / 5000 [ 16%] (Warmup)
## Chain 2 Iteration: 600 / 5000 [ 12%] (Warmup)
## Chain 3 Iteration: 600 / 5000 [ 12%] (Warmup)
## Chain 4 Iteration: 900 / 5000 [ 18%] (Warmup)
## Chain 1 Iteration: 900 / 5000 [ 18%] (Warmup)
## Chain 2 Iteration: 700 / 5000 [ 14%] (Warmup)
## Chain 3 Iteration: 700 / 5000 [ 14%] (Warmup)
## Chain 2 Iteration: 800 / 5000 [ 16%] (Warmup)
## Chain 1 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 1 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 3 Iteration: 800 / 5000 [ 16%] (Warmup)
## Chain 4 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 4 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 2 Iteration: 900 / 5000 [ 18%] (Warmup)
## Chain 1 Iteration: 1100 / 5000 [ 22%] (Sampling)
## Chain 3 Iteration: 900 / 5000 [ 18%] (Warmup)
## Chain 2 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 2 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 4 Iteration: 1100 / 5000 [ 22%] (Sampling)
## Chain 1 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 2 Iteration: 1100 / 5000 [ 22%] (Sampling)
## Chain 3 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 3 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 2 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 1 Iteration: 1300 / 5000 [ 26%] (Sampling)
## Chain 4 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 3 Iteration: 1100 / 5000 [ 22%] (Sampling)
## Chain 2 Iteration: 1300 / 5000 [ 26%] (Sampling)
## Chain 1 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 3 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 2 Iteration: 1400 / 5000 [ 28%] (Sampling)

```

```

## Chain 4 Iteration: 1300 / 5000 [ 26%] (Sampling)
## Chain 1 Iteration: 1500 / 5000 [ 30%] (Sampling)
## Chain 3 Iteration: 1300 / 5000 [ 26%] (Sampling)
## Chain 2 Iteration: 1500 / 5000 [ 30%] (Sampling)
## Chain 4 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 2 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 1 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 3 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 2 Iteration: 1700 / 5000 [ 34%] (Sampling)
## Chain 3 Iteration: 1500 / 5000 [ 30%] (Sampling)
## Chain 1 Iteration: 1700 / 5000 [ 34%] (Sampling)
## Chain 4 Iteration: 1500 / 5000 [ 30%] (Sampling)
## Chain 2 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 3 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 1 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 2 Iteration: 1900 / 5000 [ 38%] (Sampling)
## Chain 4 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 3 Iteration: 1700 / 5000 [ 34%] (Sampling)
## Chain 1 Iteration: 1900 / 5000 [ 38%] (Sampling)
## Chain 2 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 2 Iteration: 2100 / 5000 [ 42%] (Sampling)
## Chain 4 Iteration: 1700 / 5000 [ 34%] (Sampling)
## Chain 3 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 1 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 2 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 3 Iteration: 1900 / 5000 [ 38%] (Sampling)
## Chain 1 Iteration: 2100 / 5000 [ 42%] (Sampling)
## Chain 4 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 2 Iteration: 2300 / 5000 [ 46%] (Sampling)
## Chain 3 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 1 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 2 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 4 Iteration: 1900 / 5000 [ 38%] (Sampling)
## Chain 3 Iteration: 2100 / 5000 [ 42%] (Sampling)
## Chain 2 Iteration: 2500 / 5000 [ 50%] (Sampling)
## Chain 1 Iteration: 2300 / 5000 [ 46%] (Sampling)
## Chain 2 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 3 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 4 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 1 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 2 Iteration: 2700 / 5000 [ 54%] (Sampling)
## Chain 3 Iteration: 2300 / 5000 [ 46%] (Sampling)
## Chain 1 Iteration: 2500 / 5000 [ 50%] (Sampling)
## Chain 2 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 4 Iteration: 2100 / 5000 [ 42%] (Sampling)
## Chain 3 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 1 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 2 Iteration: 2900 / 5000 [ 58%] (Sampling)
## Chain 4 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 3 Iteration: 2500 / 5000 [ 50%] (Sampling)
## Chain 2 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 1 Iteration: 2700 / 5000 [ 54%] (Sampling)
## Chain 2 Iteration: 3100 / 5000 [ 62%] (Sampling)
## Chain 3 Iteration: 2600 / 5000 [ 52%] (Sampling)

```

```

## Chain 1 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 4 Iteration: 2300 / 5000 [ 46%] (Sampling)
## Chain 2 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 3 Iteration: 2700 / 5000 [ 54%] (Sampling)
## Chain 1 Iteration: 2900 / 5000 [ 58%] (Sampling)
## Chain 2 Iteration: 3300 / 5000 [ 66%] (Sampling)
## Chain 4 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 3 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 2 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 1 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 3 Iteration: 2900 / 5000 [ 58%] (Sampling)
## Chain 2 Iteration: 3500 / 5000 [ 70%] (Sampling)
## Chain 4 Iteration: 2500 / 5000 [ 50%] (Sampling)
## Chain 1 Iteration: 3100 / 5000 [ 62%] (Sampling)
## Chain 2 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 3 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 1 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 4 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 2 Iteration: 3700 / 5000 [ 74%] (Sampling)
## Chain 3 Iteration: 3100 / 5000 [ 62%] (Sampling)
## Chain 2 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 1 Iteration: 3300 / 5000 [ 66%] (Sampling)
## Chain 3 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 4 Iteration: 2700 / 5000 [ 54%] (Sampling)
## Chain 2 Iteration: 3900 / 5000 [ 78%] (Sampling)
## Chain 1 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 3 Iteration: 3300 / 5000 [ 66%] (Sampling)
## Chain 2 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 4 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 1 Iteration: 3500 / 5000 [ 70%] (Sampling)
## Chain 2 Iteration: 4100 / 5000 [ 82%] (Sampling)
## Chain 3 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 1 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 2 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 4 Iteration: 2900 / 5000 [ 58%] (Sampling)
## Chain 3 Iteration: 3500 / 5000 [ 70%] (Sampling)
## Chain 2 Iteration: 4300 / 5000 [ 86%] (Sampling)
## Chain 1 Iteration: 3700 / 5000 [ 74%] (Sampling)
## Chain 3 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 4 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 2 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 1 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 3 Iteration: 3700 / 5000 [ 74%] (Sampling)
## Chain 2 Iteration: 4500 / 5000 [ 90%] (Sampling)
## Chain 4 Iteration: 3100 / 5000 [ 62%] (Sampling)
## Chain 1 Iteration: 3900 / 5000 [ 78%] (Sampling)
## Chain 2 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 3 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 2 Iteration: 4700 / 5000 [ 94%] (Sampling)
## Chain 1 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 4 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 3 Iteration: 3900 / 5000 [ 78%] (Sampling)
## Chain 2 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 1 Iteration: 4100 / 5000 [ 82%] (Sampling)

```



```

## Chain 3 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 2 Iteration: 4900 / 5000 [ 98%] (Sampling)
## Chain 4 Iteration: 3300 / 5000 [ 66%] (Sampling)
## Chain 1 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 2 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 2 finished in 55.5 seconds.
## Chain 3 Iteration: 4100 / 5000 [ 82%] (Sampling)
## Chain 4 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 1 Iteration: 4300 / 5000 [ 86%] (Sampling)
## Chain 3 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 1 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 3 Iteration: 4300 / 5000 [ 86%] (Sampling)
## Chain 4 Iteration: 3500 / 5000 [ 70%] (Sampling)
## Chain 1 Iteration: 4500 / 5000 [ 90%] (Sampling)
## Chain 3 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 4 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 1 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 3 Iteration: 4500 / 5000 [ 90%] (Sampling)
## Chain 4 Iteration: 3700 / 5000 [ 74%] (Sampling)
## Chain 1 Iteration: 4700 / 5000 [ 94%] (Sampling)
## Chain 3 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 1 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 3 Iteration: 4700 / 5000 [ 94%] (Sampling)
## Chain 4 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 1 Iteration: 4900 / 5000 [ 98%] (Sampling)
## Chain 3 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 4 Iteration: 3900 / 5000 [ 78%] (Sampling)
## Chain 1 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 3 Iteration: 4900 / 5000 [ 98%] (Sampling)
## Chain 1 finished in 65.7 seconds.
## Chain 3 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 4 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 3 finished in 66.9 seconds.
## Chain 4 Iteration: 4100 / 5000 [ 82%] (Sampling)
## Chain 4 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 4 Iteration: 4300 / 5000 [ 86%] (Sampling)
## Chain 4 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 4 Iteration: 4500 / 5000 [ 90%] (Sampling)
## Chain 4 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 4700 / 5000 [ 94%] (Sampling)
## Chain 4 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 4 Iteration: 4900 / 5000 [ 98%] (Sampling)
## Chain 4 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 4 finished in 83.7 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 68.0 seconds.
## Total execution time: 83.9 seconds.
# print number of rhats above 1.1
s <- fit_refresh_lower$summary()
s$rhat <- as.numeric(s$rhat)

subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))[, c("variable", "rhat", "ess_bulk", "ess_tail")]

```

```
## # A tibble: 0 x 4
## # i 4 variables: variable <chr>, rhat <dbl>, ess_bulk <dbl>, ess_tail <dbl>
# print string if no rhats above 1.1
if (nrow(subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))) == 0) {
  cat("No R-hat values above 1.1\n")
}
```

No R-hat values above 1.1

```
fit_refresh_higher <- mod$sample(
  data = stan_data,
  seed = 1234,
  chains = 4,
  parallel_chains = 4,
  threads_per_chain = 1,
  iter_warmup = 1000,
  iter_sampling = 4000,
  refresh = 400,
  adapt_delta = 0.99
)
```

Running MCMC with 4 parallel chains, with 1 thread(s) per chain...

```
##
## Chain 1 Iteration: 1 / 5000 [ 0%] (Warmup)
## Chain 2 Iteration: 1 / 5000 [ 0%] (Warmup)
## Chain 3 Iteration: 1 / 5000 [ 0%] (Warmup)
## Chain 4 Iteration: 1 / 5000 [ 0%] (Warmup)
## Chain 1 Iteration: 400 / 5000 [ 8%] (Warmup)
## Chain 4 Iteration: 400 / 5000 [ 8%] (Warmup)
## Chain 2 Iteration: 400 / 5000 [ 8%] (Warmup)
## Chain 3 Iteration: 400 / 5000 [ 8%] (Warmup)
## Chain 4 Iteration: 800 / 5000 [ 16%] (Warmup)
## Chain 1 Iteration: 800 / 5000 [ 16%] (Warmup)
## Chain 3 Iteration: 800 / 5000 [ 16%] (Warmup)
## Chain 2 Iteration: 800 / 5000 [ 16%] (Warmup)
## Chain 1 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 4 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 2 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 3 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 1 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 2 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 3 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 4 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 2 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 1 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 3 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 2 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 4 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 1 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 3 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 2 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 1 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 2 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 3 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 4 Iteration: 2200 / 5000 [ 44%] (Sampling)
```

```

## Chain 2 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 3 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 1 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 4 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 2 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 3 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 1 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 2 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 3 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 4 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 1 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 2 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 3 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 2 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 2 finished in 55.1 seconds.
## Chain 1 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 4 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 3 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 1 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 3 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 4 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 3 finished in 64.4 seconds.
## Chain 1 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 1 finished in 66.3 seconds.
## Chain 4 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 4 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 4 finished in 85.2 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 67.7 seconds.
## Total execution time: 85.2 seconds.

# print number of rhats above 1.1
s <- fit_refresh_higher$summary()
s$rhat <- as.numeric(s$rhat)

subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))[, c("variable", "rhat", "ess_bulk", "ess_tail")]

## # A tibble: 0 x 4
## # i 4 variables: variable <chr>, rhat <dbl>, ess_bulk <dbl>, ess_tail <dbl>

# print string if no rhats above 1.1
if (nrow(subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))) == 0) {
  cat("No R-hat values above 1.1\n")
}

## No R-hat values above 1.1

fit_adapt_delta_lower <- mod$sample(
  data = stan_data,
  seed = 1234,
  chains = 4,
  parallel_chains = 4,
  threads_per_chain = 1,
  iter_warmup = 1000,

```

```

iter_sampling = 4000,
refresh = 200,
adapt_delta = 0.9
)

```

```
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
```

```
##
## Chain 1 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 4 Iteration:  200 / 5000 [ 4%] (Warmup)
## Chain 1 Iteration:  200 / 5000 [ 4%] (Warmup)
## Chain 3 Iteration:  200 / 5000 [ 4%] (Warmup)
## Chain 2 Iteration:  200 / 5000 [ 4%] (Warmup)
## Chain 1 Iteration:  400 / 5000 [ 8%] (Warmup)
## Chain 3 Iteration:  400 / 5000 [ 8%] (Warmup)
## Chain 4 Iteration:  400 / 5000 [ 8%] (Warmup)
## Chain 2 Iteration:  400 / 5000 [ 8%] (Warmup)
## Chain 3 Iteration:  600 / 5000 [12%] (Warmup)
## Chain 4 Iteration:  600 / 5000 [12%] (Warmup)
## Chain 1 Iteration:  600 / 5000 [12%] (Warmup)
## Chain 2 Iteration:  600 / 5000 [12%] (Warmup)
## Chain 3 Iteration:  800 / 5000 [16%] (Warmup)
## Chain 4 Iteration:  800 / 5000 [16%] (Warmup)
## Chain 1 Iteration:  800 / 5000 [16%] (Warmup)
## Chain 2 Iteration:  800 / 5000 [16%] (Warmup)
## Chain 3 Iteration: 1000 / 5000 [20%] (Warmup)
## Chain 3 Iteration: 1001 / 5000 [20%] (Sampling)
## Chain 4 Iteration: 1000 / 5000 [20%] (Warmup)
## Chain 1 Iteration: 1000 / 5000 [20%] (Warmup)
## Chain 4 Iteration: 1001 / 5000 [20%] (Sampling)
## Chain 1 Iteration: 1001 / 5000 [20%] (Sampling)
## Chain 2 Iteration: 1000 / 5000 [20%] (Warmup)
## Chain 2 Iteration: 1001 / 5000 [20%] (Sampling)
## Chain 1 Iteration: 1200 / 5000 [24%] (Sampling)
## Chain 3 Iteration: 1200 / 5000 [24%] (Sampling)
## Chain 4 Iteration: 1200 / 5000 [24%] (Sampling)
## Chain 2 Iteration: 1200 / 5000 [24%] (Sampling)
## Chain 1 Iteration: 1400 / 5000 [28%] (Sampling)
## Chain 3 Iteration: 1400 / 5000 [28%] (Sampling)
## Chain 2 Iteration: 1400 / 5000 [28%] (Sampling)
## Chain 4 Iteration: 1400 / 5000 [28%] (Sampling)
## Chain 1 Iteration: 1600 / 5000 [32%] (Sampling)
## Chain 3 Iteration: 1600 / 5000 [32%] (Sampling)
## Chain 2 Iteration: 1600 / 5000 [32%] (Sampling)
## Chain 1 Iteration: 1800 / 5000 [36%] (Sampling)
## Chain 4 Iteration: 1600 / 5000 [32%] (Sampling)
## Chain 3 Iteration: 1800 / 5000 [36%] (Sampling)
## Chain 2 Iteration: 1800 / 5000 [36%] (Sampling)
## Chain 1 Iteration: 2000 / 5000 [40%] (Sampling)
## Chain 3 Iteration: 2000 / 5000 [40%] (Sampling)
## Chain 4 Iteration: 1800 / 5000 [36%] (Sampling)
## Chain 2 Iteration: 2000 / 5000 [40%] (Sampling)

```

```

## Chain 1 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 3 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 2 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 4 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 1 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 3 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 2 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 1 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 4 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 3 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 2 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 1 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 3 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 4 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 1 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 2 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 3 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 1 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 4 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 2 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 3 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 1 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 2 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 4 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 3 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 1 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 2 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 4 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 1 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 3 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 2 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 1 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 3 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 4 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 2 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 1 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 3 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 2 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 4 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 1 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 3 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 2 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 1 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 3 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 2 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 1 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 3 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 2 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 1 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 1 finished in 32.4 seconds.
## Chain 2 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 3 Iteration: 4800 / 5000 [ 96%] (Sampling)

```

```

## Chain 4 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 3 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 3 finished in 34.4 seconds.
## Chain 2 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 2 finished in 34.6 seconds.
## Chain 4 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 4 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 4 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 4 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 4 finished in 41.9 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 35.8 seconds.
## Total execution time: 42.0 seconds.

# print number of rhats above 1.1
s <- fit_adapt_delta_lower$summary()
s$rhat <- as.numeric(s$rhat)

subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))[, c("variable", "rhat", "ess_bulk", "ess_tail")]

## # A tibble: 0 x 4
## # i 4 variables: variable <chr>, rhat <dbl>, ess_bulk <dbl>, ess_tail <dbl>

# print string if no rhats above 1.1
if (nrow(subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))) == 0) {
  cat("No R-hat values above 1.1\n")
}

## No R-hat values above 1.1

fit_adapt_delta_higher <- mod$sample(
  data = stan_data,
  seed = 1234,
  chains = 4,
  parallel_chains = 4,
  threads_per_chain = 1,
  iter_warmup = 1000,
  iter_sampling = 4000,
  refresh = 200,
  adapt_delta = 0.999
)

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 5000 [ 0%] (Warmup)
## Chain 1 Iteration:  200 / 5000 [ 4%] (Warmup)
## Chain 2 Iteration:  200 / 5000 [ 4%] (Warmup)
## Chain 3 Iteration:  200 / 5000 [ 4%] (Warmup)
## Chain 1 Iteration:  400 / 5000 [ 8%] (Warmup)
## Chain 2 Iteration:  400 / 5000 [ 8%] (Warmup)
## Chain 1 Iteration:  600 / 5000 [12%] (Warmup)

```

```

## Chain 3 Iteration: 400 / 5000 [ 8%] (Warmup)
## Chain 4 Iteration: 200 / 5000 [ 4%] (Warmup)
## Chain 1 Iteration: 800 / 5000 [ 16%] (Warmup)
## Chain 2 Iteration: 600 / 5000 [ 12%] (Warmup)
## Chain 3 Iteration: 600 / 5000 [ 12%] (Warmup)
## Chain 2 Iteration: 800 / 5000 [ 16%] (Warmup)
## Chain 3 Iteration: 800 / 5000 [ 16%] (Warmup)
## Chain 4 Iteration: 400 / 5000 [ 8%] (Warmup)
## Chain 1 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 1 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 2 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 2 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 4 Iteration: 600 / 5000 [ 12%] (Warmup)
## Chain 3 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 3 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 4 Iteration: 800 / 5000 [ 16%] (Warmup)
## Chain 2 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 1 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 2 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 4 Iteration: 1000 / 5000 [ 20%] (Warmup)
## Chain 4 Iteration: 1001 / 5000 [ 20%] (Sampling)
## Chain 3 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 2 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 1 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 4 Iteration: 1200 / 5000 [ 24%] (Sampling)
## Chain 2 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 3 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 4 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 1 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 2 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 3 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 4 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 2 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 1 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 2 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 4 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 3 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 2 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 1 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 4 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 2 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 3 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 4 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 1 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 2 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 3 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 4 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 2 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 1 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 2 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 4 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 3 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 2 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 1 Iteration: 2600 / 5000 [ 52%] (Sampling)

```

```

## Chain 4 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 2 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 3 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 4 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 2 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 1 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 4 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 2 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 3 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 1 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 2 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 4 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 3 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 2 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 1 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 2 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 3 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 4 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 2 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 2 finished in 85.9 seconds.
## Chain 1 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 4 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 3 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 1 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 4 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 3 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 4 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 1 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 3 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 4 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 1 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 4 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 3 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 1 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 4 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 4 finished in 110.3 seconds.
## Chain 3 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 1 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 3 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 1 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 3 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 1 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 3 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 1 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 1 finished in 131.9 seconds.
## Chain 3 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 3 finished in 135.5 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 115.9 seconds.
## Total execution time: 135.6 seconds.

```



```

# print number of rhats above 1.1
s <- fit_adapt_delta_higher$summary()
s$rhat <- as.numeric(s$rhat)

subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))[, c("variable", "rhat", "ess_bulk", "ess_tail")]

## # A tibble: 0 x 4
## # i 4 variables: variable <chr>, rhat <dbl>, ess_bulk <dbl>, ess_tail <dbl>
# print string if no rhats above 1.1
if (nrow(subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))) == 0) {
  cat("No R-hat values above 1.1\n")
}

## No R-hat values above 1.1
if (interactive() && is.null(knitr::opts_knit$get("rmarkdown.pandoc.to"))) {
  library(shinystan)
  launch_shinystan(fit_basis)
}

if (interactive() && is.null(knitr::opts_knit$get("rmarkdown.pandoc.to"))) {
  library(shinystan)
  launch_shinystan(fit_adapt_delta_lower)
}

```

Robustness check - Are there differences in the results if one changes convergence settings?

```

library(posterior)
names(summarise_draws(as_draws_df(fit_basis$draws("pi")), mean, sd, ~quantile(.x, c(.05, .95))))

## [1] "variable" "mean"      "sd"        "5%"        "95%"

compare_param <- function(fit1, fit2, var) {
  d1 <- as_draws_df(fit1$draws(var))
  d2 <- as_draws_df(fit2$draws(var))

  s1 <- summarise_draws(d1, mean, sd, ~quantile(.x, c(.05, .95), na.rm = TRUE))
  s2 <- summarise_draws(d2, mean, sd, ~quantile(.x, c(.05, .95), na.rm = TRUE))

  out <- s1[, c("variable", "mean", "sd", "5%", "95%")]
  out$mean_2 <- s2$mean
  out$sd_2 <- s2$sd
  out$`5%_2` <- s2$`5%`
  out$`95%_2` <- s2$`95%`
  out$diff_mean <- out$mean_2 - out$mean
  out
}

# 1) Compare parameters
pi_cmp <- compare_param(fit_basis, fit_adapt_delta_lower, "pi")
theta_cmp <- compare_param(fit_basis, fit_adapt_delta_lower, "theta")

```

#2) Show largest differences

```
pi_cmp[order(abs(pi_cmp$diff_mean), decreasing = TRUE), ]
```

```
## # A tibble: 2 x 10
```

```
##   variable mean      sd '5%' '95%' mean_2 sd_2 '5%_2' '95%_2' diff_mean
##   <chr>      <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 pi[1]      0.478 0.0229 0.441 0.516 0.478 0.0231 0.441 0.516 -0.000229
## 2 pi[2]      0.522 0.0229 0.484 0.559 0.522 0.0231 0.484 0.559 0.000229
```

```
theta_cmp[order(abs(theta_cmp$diff_mean), decreasing = TRUE), ][1:20, ]
```

```
## # A tibble: 20 x 10
```

```
##   variable      mean      sd      '5%'      '95%' mean_2 sd_2 '5%_2' '95%_2' diff_mean
##   <chr>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl> <dbl> <dbl> <dbl>
## 1 theta[3,2,2] 0.980    0.00887 0.964    0.992    0.980 0.00881 0.964    0.992 -0.0000731
## 2 theta[3,2,1] 0.0201   0.00887 0.00800 0.0363   0.0202 0.00881 0.00810 0.0363 0.0000731
## 3 theta[3,1,1] 0.996    0.00433 0.987    1.00     0.996 0.00446 0.987    1.00 -0.0000278
## 4 theta[3,1,2] 0.00441   0.00433 0.000228 0.0131   0.00444 0.00446 0.000219 0.0133 0.0000278
## 5 theta[2,1,1] 0.996    0.00438 0.987    1.00     0.996 0.00451 0.986    1.00 -0.0000273
## 6 theta[2,1,2] 0.00440   0.00438 0.000221 0.0132   0.00443 0.00451 0.000230 0.0136 0.0000273
## 7 theta[1,2,1] 0.00405   0.00401 0.000201 0.0121   0.00403 0.00400 0.000216 0.0119 -0.0000221
## 8 theta[1,2,2] 0.996    0.00401 0.988    1.00     0.996 0.00400 0.988    1.00 0.0000221
## 9 theta[2,2,2] 0.996    0.00407 0.988    1.00     0.996 0.00399 0.988    1.00 0.0000209
## 10 theta[2,2,1] 0.00404   0.00407 0.000198 0.0121   0.00402 0.00399 0.000197 0.0121 -0.0000209
## 11 theta[1,1,1] 0.996    0.00447 0.987    1.00     0.996 0.00444 0.987    1.00 -0.00000746
## 12 theta[1,1,2] 0.00442   0.00447 0.000220 0.0134   0.00443 0.00444 0.000214 0.0135 0.00000745
## 13 <NA>        NA        NA        NA        NA        NA        NA        NA        NA        NA
## 14 <NA>        NA        NA        NA        NA        NA        NA        NA        NA        NA
## 15 <NA>        NA        NA        NA        NA        NA        NA        NA        NA        NA
## 16 <NA>        NA        NA        NA        NA        NA        NA        NA        NA        NA
## 17 <NA>        NA        NA        NA        NA        NA        NA        NA        NA        NA
## 18 <NA>        NA        NA        NA        NA        NA        NA        NA        NA        NA
## 19 <NA>        NA        NA        NA        NA        NA        NA        NA        NA        NA
## 20 <NA>        NA        NA        NA        NA        NA        NA        NA        NA        NA
```

show only differences larger than 0.001

```
subset(theta_cmp, abs(diff_mean) > 1e-3)
```

```
## # A tibble: 0 x 10
```

```
## # i 10 variables: variable <chr>, mean <dbl>, sd <dbl>, 5% <dbl>, 95% <dbl>, mean_2 <dbl>, sd_2 <dbl>,
## # 5%_2 <dbl>, 95%_2 <dbl>, diff_mean <dbl>
```

no content

```
theta_cmp$overlap_5_95 <- pmin(theta_cmp$`95%`, theta_cmp$`95%_2`) -
  pmax(theta_cmp$`5%`, theta_cmp$`5%_2`)
```

negative values mean no overlap

```
theta_cmp[order(theta_cmp$overlap_5_95), ][1:20, ]
```

```
## # A tibble: 20 x 11
```

```
##   variable      mean      sd      '5%'      '95%' mean_2 sd_2 '5%_2' '95%_2' diff_mean
##   <chr>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl> <dbl> <dbl> <dbl>
## 1 theta[1,2,2] 0.996    0.00401 0.988    1.00     0.996 0.00400 0.988    1.00 0.0000221
## 2 theta[1,2,1] 0.00405   0.00401 0.000201 0.0121   0.00403 0.00400 0.000216 0.0119 -0.0000221
## 3 theta[2,2,2] 0.996    0.00407 0.988    1.00     0.996 0.00399 0.988    1.00 0.0000209
```

```
## 4 theta[2,2,1] 0.00404 0.00407 0.000198 0.0121 0.00402 0.00399 0.000197 0.0121 -0.0000209
## 5 theta[3,1,2] 0.00441 0.00433 0.000228 0.0131 0.00444 0.00446 0.000219 0.0133 0.0000278
## 6 theta[3,1,1] 0.996 0.00433 0.987 1.00 0.996 0.00446 0.987 1.00 -0.0000278
## 7 theta[2,1,2] 0.00440 0.00438 0.000221 0.0132 0.00443 0.00451 0.000230 0.0136 0.0000273
## 8 theta[2,1,1] 0.996 0.00438 0.987 1.00 0.996 0.00451 0.986 1.00 -0.0000273
## 9 theta[1,1,2] 0.00442 0.00447 0.000220 0.0134 0.00443 0.00444 0.000214 0.0135 0.00000745
## 10 theta[1,1,1] 0.996 0.00447 0.987 1.00 0.996 0.00444 0.987 1.00 -0.00000746
## 11 theta[3,2,2] 0.980 0.00887 0.964 0.992 0.980 0.00881 0.964 0.992 -0.0000731
## 12 theta[3,2,1] 0.0201 0.00887 0.00800 0.0363 0.0202 0.00881 0.00810 0.0363 0.0000731
## 13 <NA> NA NA NA NA NA NA NA NA NA
## 14 <NA> NA NA NA NA NA NA NA NA NA
## 15 <NA> NA NA NA NA NA NA NA NA NA
## 16 <NA> NA NA NA NA NA NA NA NA NA
## 17 <NA> NA NA NA NA NA NA NA NA NA
## 18 <NA> NA NA NA NA NA NA NA NA NA
## 19 <NA> NA NA NA NA NA NA NA NA NA
## 20 <NA> NA NA NA NA NA NA NA NA NA
```

```
theta_cmp$diff_over_sd <- abs(theta_cmp$diff_mean) / theta_cmp$sd
theta_cmp[order(theta_cmp$diff_over_sd, decreasing = TRUE), ][1:20, ]
```

```
## # A tibble: 20 x 12
##   variable      mean      sd      '5%'      '95%'   mean_2   sd_2      '5%_2'      '95%_2'  diff_mean
##   <chr>      <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
## 1 theta[3,2,2] 0.980 0.00887 0.964 0.992 0.980 0.00881 0.964 0.992 -0.0000731
## 2 theta[3,2,1] 0.0201 0.00887 0.00800 0.0363 0.0202 0.00881 0.00810 0.0363 0.0000731
## 3 theta[3,1,1] 0.996 0.00433 0.987 1.00 0.996 0.00446 0.987 1.00 -0.0000278
## 4 theta[3,1,2] 0.00441 0.00433 0.000228 0.0131 0.00444 0.00446 0.000219 0.0133 0.0000278
## 5 theta[2,1,1] 0.996 0.00438 0.987 1.00 0.996 0.00451 0.986 1.00 -0.0000273
## 6 theta[2,1,2] 0.00440 0.00438 0.000221 0.0132 0.00443 0.00451 0.000230 0.0136 0.0000273
## 7 theta[1,2,1] 0.00405 0.00401 0.000201 0.0121 0.00403 0.00400 0.000216 0.0119 -0.0000221
## 8 theta[1,2,2] 0.996 0.00401 0.988 1.00 0.996 0.00400 0.988 1.00 0.0000221
## 9 theta[2,2,2] 0.996 0.00407 0.988 1.00 0.996 0.00399 0.988 1.00 0.0000209
## 10 theta[2,2,1] 0.00404 0.00407 0.000198 0.0121 0.00402 0.00399 0.000197 0.0121 -0.0000209
## 11 theta[1,1,1] 0.996 0.00447 0.987 1.00 0.996 0.00444 0.987 1.00 -0.00000746
## 12 theta[1,1,2] 0.00442 0.00447 0.000220 0.0134 0.00443 0.00444 0.000214 0.0135 0.00000745
## 13 <NA> NA NA NA NA NA NA NA NA NA
## 14 <NA> NA NA NA NA NA NA NA NA NA
## 15 <NA> NA NA NA NA NA NA NA NA NA
## 16 <NA> NA NA NA NA NA NA NA NA NA
## 17 <NA> NA NA NA NA NA NA NA NA NA
## 18 <NA> NA NA NA NA NA NA NA NA NA
## 19 <NA> NA NA NA NA NA NA NA NA NA
## 20 <NA> NA NA NA NA NA NA NA NA NA
## # i 1 more variable: diff_over_sd <dbl>
```

```
# library(dplyr)
# library(posterior)
# library(ggplot2)
# library(reshape2)
#
# out_dir <- "img/model_results/kladb/two_categories"
# dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)
#
# save_plot <- function(p, filename, width = 7, height = 5, dpi = 600) {
```

```

#   ggsave(
#     filename = file.path(out_dir, filename),
#     plot = p,
#     width = width,
#     height = height,
#     dpi = dpi
#   )
# }
#
# # -----
# # 0) Basic diagnostics for key parameters
# # -----
# diag_tbl <- fit_basis$summary(variables = c("pi", "theta")) %>%
#   dplyr::select(variable, rhat, ess_bulk, ess_tail)
#
# diag_bad <- diag_tbl %>%
#   dplyr::filter(is.finite(rhat), rhat > 1.01) %>%
#   dplyr::arrange(desc(rhat))
#
# diag_bad
#
# # Divergences, treedepth hits, if available (cmdstanr)
# div_info <- tryCatch(fit_basis$diagnostic_summary(), error = function(e) NULL)
# if (is.null(div_info)) {
#   div_info <- tryCatch(fit_basis$cmdstan_diagnose(), error = function(e) NULL)
# }
# div_info
#
# # -----
# # 1) Confusion matrices (theta), posterior mean and uncertainty
# # -----
# theta_draws_mat <- fit_basis$draws("theta", format = "draws_matrix")
# theta_mean_vec <- colMeans(theta_draws_mat)
#
# # Expectation: theta has dimensions [J rater, K true, K given]
# theta_mean_arr <- array(
#   theta_mean_vec,
#   dim = c(J, K, K),
#   dimnames = list(
#     rater = c("human1", "human2", "tool"),
#     true = kldb_levels,
#     given = kldb_levels
#   )
# )
#
# theta_human1_mean <- theta_mean_arr["human1", , ]
# theta_human2_mean <- theta_mean_arr["human2", , ]
# theta_tool_mean <- theta_mean_arr["tool", , ]
#
# theta_human1_mean
# rowSums(theta_human1_mean)
#
# theta_human2_mean

```

```

# rowSums(theta_human2_mean)
#
# theta_tool_mean
# rowSums(theta_tool_mean)
#
# plot_theta <- function(mat, title_txt) {
#   long <- reshape2::melt(mat, varnames = c("True", "Given"), value.name = "Prob")
#   ggplot(long, aes(x = Given, y = True, fill = Prob)) +
#     geom_tile(color = "white") +
#     scale_fill_gradient(low = "white", high = "darkgreen") +
#     theme_light() +
#     labs(
#       title = title_txt,
#       x = "Observed class",
#       y = "True class C_i"
#     ) +
#     theme(axis.text.x = element_text(angle = 45, hjust = 1))
# }
#
# p_theta_h1_mean <- plot_theta(theta_human1_mean, "Confusion matrix  $\pi^{(1)}$  (posterior mean)")
# p_theta_h2_mean <- plot_theta(theta_human2_mean, "Confusion matrix  $\pi^{(2)}$  (posterior mean)")
# p_theta_tool_mean <- plot_theta(theta_tool_mean, "Confusion matrix  $\pi^{(3)}$  (posterior mean)")
#
# save_plot(p_theta_h1_mean, "theta_human1_mean.png")
# save_plot(p_theta_h2_mean, "theta_human2_mean.png")
# save_plot(p_theta_tool_mean, "theta_tool_mean.png")
#
# p_theta_h1_mean
# p_theta_h2_mean
# p_theta_tool_mean
#
# # Credible intervals for all theta entries
# theta_summ <- summarise_draws(
#   as_draws_df(fit_basis$draws("theta")),
#   mean, sd,
#   ~quantile(.x, probs = c(`5%` = 0.05, `50%` = 0.50, `95%` = 0.95), na.rm = TRUE)
# )
# theta_summ
#
# # Posterior SD heatmaps for theta (uncertainty by cell)
# theta_sd_vec <- apply(theta_draws_mat, 2, sd)
# theta_sd_arr <- array(
#   theta_sd_vec,
#   dim = c(J, K, K),
#   dimnames = list(
#     rater = c("human1", "human2", "tool"),
#     true = kldb_levels,
#     given = kldb_levels
#   )
# )
#
# p_theta_h1_sd <- plot_theta(theta_sd_arr["human1", , ], "Confusion matrix  $\pi^{(1)}$  (posterior SD)")
# p_theta_h2_sd <- plot_theta(theta_sd_arr["human2", , ], "Confusion matrix  $\pi^{(2)}$  (posterior SD)")

```

```

# p_theta_tool_sd <- plot_theta(theta_sd_arr["tool", , ], "Confusion matrix  $\pi^3$  (posterior SD)")
#
# save_plot(p_theta_h1_sd, "theta_human1_sd.png")
# save_plot(p_theta_h2_sd, "theta_human2_sd.png")
# save_plot(p_theta_tool_sd, "theta_tool_sd.png")
#
# p_theta_h1_sd
# p_theta_h2_sd
# p_theta_tool_sd
#
# # -----
# # 2) Class prevalence ( $\pi$ ), posterior mean and uncertainty
# # -----
# pi_summ <- summarise_draws(
#   as_draws_df(fit_basis$draws("pi")),
#   mean, sd,
#   ~quantile(.x, probs = c(`5%` = 0.05, `50%` = 0.50, `95%` = 0.95), na.rm = TRUE)
# )
# pi_summ
#
# pi_means <- colMeans(as_draws_matrix(fit_basis$draws("pi")))
# pi_df <- data.frame(
#   class = kldb_levels,
#   prevalence = as.numeric(pi_means)
# )
#
# p_pi_bar <- ggplot(pi_df, aes(x = class, y = prevalence)) +
#   geom_bar(stat = "identity", fill = "#02626f") +
#   theme_light() +
#   labs(
#     title = "Posterior mean of class prevalence ( $\pi$ )",
#     x = "Class",
#     y = "Prevalence"
#   ) +
#   theme(axis.text.x = element_text(angle = 45, hjust = 1))
#
# save_plot(p_pi_bar, "pi_mean_bar.png", width = 6, height = 4)
# p_pi_bar
#
# # -----
# # 2b) Rater level scalar summaries from theta
# # -----
# pi_mean_vec <- as.numeric(pi_means)
# names(pi_mean_vec) <- kldb_levels
#
# weighted_diag_mass <- function(mat, w) sum(diag(mat) * w)
# weighted_offdiag_mass <- function(mat, w) sum((1 - diag(mat)) * w)
#
# rater_summary <- data.frame(
#   rater = c("human1", "human2", "tool"),
#   weighted_diag_mass = c(
#     weighted_diag_mass(theta_human1_mean, pi_mean_vec),
#     weighted_diag_mass(theta_human2_mean, pi_mean_vec),

```

```

#   weighted_diag_mass(theta_tool_mean,   pi_mean_vec)
# ),
#   weighted_offdiag_mass = c(
#     weighted_offdiag_mass(theta_human1_mean, pi_mean_vec),
#     weighted_offdiag_mass(theta_human2_mean, pi_mean_vec),
#     weighted_offdiag_mass(theta_tool_mean,   pi_mean_vec)
#   )
# )
# rater_summary
#
# p_rater_diag <- ggplot(rater_summary, aes(x = rater, y = weighted_diag_mass)) +
#   geom_bar(stat = "identity", fill = "#02626f") +
#   theme_light() +
#   labs(
#     title = "Rater summary: Weighted diagonal mass",
#     x = "Rater",
#     y = "Weighted diagonal mass"
#   )
#
# save_plot(p_rater_diag, "rater_weighted_diag_mass.png", width = 6, height = 4)
# p_rater_diag
#
# # -----
# # 3) Posterior class probabilities per item (q_z)
# # -----
# qz_draws_mat <- fit_basis$draws("q_z", format = "draws_matrix")
# qz_mean_vec <- colMeans(qz_draws_mat)
#
# qz_mean <- matrix(
#   qz_mean_vec,
#   nrow = I, ncol = K,
#   dimnames = list(item = item_ids, class = kldb_levels)
# )
#
# qz_df <- as.data.frame(qz_mean)
# p_cols <- paste0("p_", kldb_levels)
# colnames(qz_df) <- p_cols
# qz_df$item_id <- item_ids
#
# # MAP label
# qz_df$map_class_idx <- max.col(qz_df[p_cols], ties.method = "first")
# qz_df$map_class_label <- kldb_levels[qz_df$map_class_idx]
#
# # Uncertainty measures
# eps <- 1e-12
# qz_df$max_p <- apply(qz_df[p_cols], 1, max)
#
# qz_df$entropy <- -rowSums(as.matrix(qz_df[p_cols]) * log(as.matrix(qz_df[p_cols]) + eps))
#
# qz_df$margin <- apply(qz_df[p_cols], 1, function(p) {
#   ps <- sort(p, decreasing = TRUE)
#   ps[1] - ps[2]
# })

```

```

#
# # Credible set per item, smallest set reaching cumulative posterior mass >= level
# credible_set <- function(p, classes, level = 0.95) {
#   o <- order(p, decreasing = TRUE)
#   cum <- cumsum(p[o])
#   m <- which(cum >= level)[1]
#   classes[o[seq_len(m)]]
# }
# qz_df$cs95 <- apply(qz_df[p_cols], 1, function(x) credible_set(x, kldb_levels, level = 0.95))
#
# table(lengths(qz_df$cs95))
#
# # Plots of uncertainty and confidence
# p_maxp_hist <- ggplot(qz_df, aes(x = max_p)) +
#   geom_histogram(binwidth = 0.025, fill = "#333333", color = "white", size = 0.1) +
#   theme_light() +
#   labs(x = "Posterior max probability", y = "Count",
#        title = "Model confidence per item")
#
# p_entropy_hist <- ggplot(qz_df, aes(x = entropy)) +
#   geom_histogram(binwidth = 0.025, fill = "#333333", color = "white", size = 0.1) +
#   theme_light() +
#   labs(x = "Posterior entropy", y = "Count",
#        title = "Item uncertainty (entropy)")
#
# p_margin_hist <- ggplot(qz_df, aes(x = margin)) +
#   geom_histogram(binwidth = 0.025, fill = "#333333", color = "white", size = 0.1) +
#   theme_light() +
#   labs(x = "Posterior margin (top1 minus top2)", y = "Count",
#        title = "Item uncertainty (margin)")
#
# save_plot(p_maxp_hist, "qz_maxp_hist.png", width = 6, height = 4)
# save_plot(p_entropy_hist, "qz_entropy_hist.png", width = 6, height = 4)
# save_plot(p_margin_hist, "qz_margin_hist.png", width = 6, height = 4)
#
# p_maxp_hist
# p_entropy_hist
# p_margin_hist
#
# # -----
# # 3b) MAP distribution vs pi (sanity check)
# # -----
# map_tab <- qz_df %>%
#   count(map_class_label, name = "n_map") %>%
#   mutate(map_share = n_map / sum(n_map)) %>%
#   rename(class = map_class_label)
#
# pi_tab <- data.frame(class = kldb_levels, pi_mean = as.numeric(pi_means))
#
# map_vs_pi <- left_join(map_tab, pi_tab, by = "class")
# map_vs_pi
#
# p_map_vs_pi <- ggplot(map_vs_pi, aes(x = class)) +

```



```

#   geom_col(aes(y = map_share), fill = "#02626f") +
#   geom_point(aes(y = pi_mean), size = 2) +
#   theme_light() +
#   labs(
#     title = "MAP share (bars) vs posterior mean p (points)",
#     x = "Class",
#     y = "Share / prevalence"
#   )
#
# save_plot(p_map_vs_pi, "map_share_vs_pi.png", width = 6, height = 4)
# p_map_vs_pi
#
# # -----
# # 4) Merge predictions with original ratings, include uncertainty
# # -----
# ratings <- kldb_data %>%
#   dplyr::select(item_id, kldb_human1, kldb_human2, kldb_tool)
#
# predictions <- qz_df %>%
#   dplyr::select(item_id, map_class_label, all_of(p_cols), max_p, entropy, margin, cs95)
#
# ratings_with_predictions <- ratings %>%
#   dplyr::left_join(predictions, by = "item_id") %>%
#   dplyr::rename(
#     predicted_class = map_class_label,
#     rating_human1 = kldb_human1,
#     rating_human2 = kldb_human2,
#     rating_tool = kldb_tool
#   ) %>%
#   dplyr::arrange(item_id)
#
# head(ratings_with_predictions, 30)
#
# # Most uncertain items by entropy, include posterior probabilities
# most_uncertain <- ratings_with_predictions %>%
#   dplyr::arrange(desc(entropy)) %>%
#   dplyr::select(item_id, predicted_class, entropy, margin, max_p, all_of(p_cols),
#     rating_human1, rating_human2, rating_tool)
#
# head(most_uncertain, 20)
#
# # -----
# # 5) Observed agreement, descriptive, not model based
# # -----
# mean(kldb_data$kldb_human1 == kldb_data$kldb_tool, na.rm = TRUE)
# mean(kldb_data$kldb_human2 == kldb_data$kldb_tool, na.rm = TRUE)
#
# kldb_data %>%
#   dplyr::count(kldb_human1, kldb_human2, kldb_tool) %>%
#   dplyr::arrange(desc(n))
#
# # 5) Posterior summaries
# # Confusion matrices per rater (posterior mean)
# theta_draws_rvars <- fit_adapt_delta_lower$draws("theta")

```

```

# theta_rvars <- posterior::as_draws_rvars(theta_draws_rvars)
# theta_mean_rvar <- posterior::rvar_mean(theta_rvars$theta)
#
# # Get draws directly as a matrix
# theta_draws_mat <- fit_adapt_delta_lower$draws("theta", format = "draws_matrix")
#
# # Mean across all draws
# theta_mean_vec <- colMeans(theta_draws_mat)
#
# # Reshape into a 3D array (J x K x K)
# theta_mean_arr <- array(
#   theta_mean_vec,
#   dim = c(J, K, K),
#   dimnames = list(
#     rater = c("human1", "human2", "tool"),
#     true = kldb_levels,
#     given = kldb_levels
#   )
# )
#
# # Example: Human 1 (rater 1)
# theta_human1_mean <- theta_mean_arr["human1", , ]
# theta_human1_mean
#
# rowSums(theta_human1_mean)
# # Should all be about 1
#
# # Same for rater 2 and tool
# theta_human2_mean <- theta_mean_arr["human2", , ]
# theta_tool_mean <- theta_mean_arr["tool", , ]
#
# theta_human2_mean
# theta_tool_mean
#
# # Posterior class probabilities per item (with original labels)
# qz_draws_array <- fit_adapt_delta_lower$draws("q_z", format = "draws_array") # [iter, chain, I, K]
# qz_draws_mat <- fit_adapt_delta_lower$draws("q_z", format = "draws_matrix")
#
# # Mean across all draws
# qz_mean_vec <- colMeans(qz_draws_mat)
#
# # Reshape into an I x K matrix
# qz_mean <- matrix(
#   qz_mean_vec,
#   nrow = I, ncol = K,
#   dimnames = list(item = item_ids, class = kldb_levels)
# )
#
# qz_df <- as.data.frame(qz_mean)
# colnames(qz_df) <- paste0("p_", kldb_levels)
# qz_df$item_id <- item_ids
#
# # MAP, both as index and as original label

```

```

# qz_df$map_class_idx <- max.col(qz_df[paste0("p_", kldb_levels)], ties.method = "first")
# qz_df$map_class_label <- kldb_levels[qz_df$map_class_idx]
#
# # Join back to items
# result_items <- data.frame(item_id = item_ids) %>%
#   dplyr::left_join(qz_df, by = "item_id")
#
# head(result_items)
# print(head(result_items, 20))
#
# kldb_data %>%
#   dplyr::count(kldb_human1, kldb_human2, kldb_tool) %>%
#   dplyr::arrange(desc(n))
#
# mean(kldb_data$kldb_human1 == kldb_data$kldb_tool, na.rm = TRUE)
# mean(kldb_data$kldb_human2 == kldb_data$kldb_tool, na.rm = TRUE)
#
# library(ggplot2)
# qz_df$max_p <- apply(qz_df[paste0("p_", kldb_levels)], 1, max)
#
# ggplot(qz_df, aes(x = max_p)) +
#   geom_histogram(binwidth = 0.025, fill = "#333333", color = "white", size = 0.1) +
#   labs(x = "Posterior max probability", y = "Count",
#        title = "Model confidence per item")
#
# # Output from already executed code
#
# rowSums(theta_human1_mean)
# # Should all be about 1
#
# library(reshape2)
# theta_human1_long <- melt(theta_human1_mean, varnames = c("True", "Given"), value.name = "Prob")
#
# ggplot(theta_human1_long, aes(x = Given, y = True, fill = Prob)) +
#   geom_tile(color = "white") +
#   scale_fill_gradient(low = "white", high = "darkgreen") +
#   theme_light() +
#   labs(
#     title = "Confusion matrix (Human rater 1)",
#     x = "Observed class",
#     y = "True class (latent)"
#   ) +
#   theme(axis.text.x = element_text(angle = 45, hjust = 1))
#
# library(posterior)
# pi_means <- colMeans(as_draws_matrix(fit_adapt_delta_lower$draws("pi")))
# pi_means
#
# kldb_levels[1]
#
# # Visualize pi_means in a bar chart
# pi_df <- data.frame(
#   class = kldb_levels,

```

```

# prevalence = pi_means
# )
#
# ggplot(pi_df, aes(x = class, y = prevalence)) +
#   geom_bar(stat = "identity", fill = "#02626f") +
#   theme_light() +
#   labs(
#     title = "Posterior mean of class prevalence (p)",
#     x = "Class",
#     y = "Prevalence"
#   ) +
#   theme(axis.text.x = element_text(angle = 45, hjust = 1))
#
# # Merge ratings and predicted class
#
# # 1) Get the original rater labels again
# ratings <- kldb_data %>%
#   dplyr::select(item_id, kldb_human1, kldb_human2, kldb_tool)
#
# # 2) Combine with posterior class assignments
# predictions <- qz_df %>%
#   dplyr::select(item_id, map_class_label, starts_with("p_"))
#
# # 3) Merge
# ratings_with_predictions <- ratings %>%
#   dplyr::left_join(predictions, by = "item_id") %>%
#   dplyr::arrange(item_id)
#
# # 4) Readable column names
# ratings_with_predictions <- ratings_with_predictions %>%
#   dplyr::rename(
#     predicted_class = map_class_label,
#     rating_human1 = kldb_human1,
#     rating_human2 = kldb_human2,
#     rating_tool = kldb_tool
#   )
#
# # 5) Example output
# head(ratings_with_predictions, 30)

```